



Practical Course - Operating Systems - seL4 & TRENTO

W1 – ARM/RISC-V, SoC, SBC and Bootloader

Copyright Notice

Copyright

- © 2021 HENSOLDT Cyber GmbH – All rights reserved.

Trademarks

- TRENTOS® is a trademark of HENSOLDT Cyber GmbH.
- seL4® is a trademark of LF Projects, LLC.

ARM

ARM today – Smartphones, Automotive, ...



Artificial Intelligence in every device



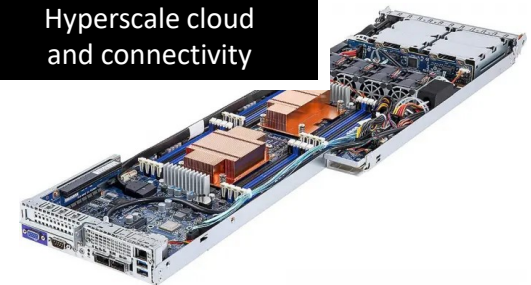
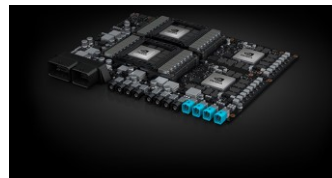
Autonomous machines



Augmented reality



Hyperscale cloud and connectivity

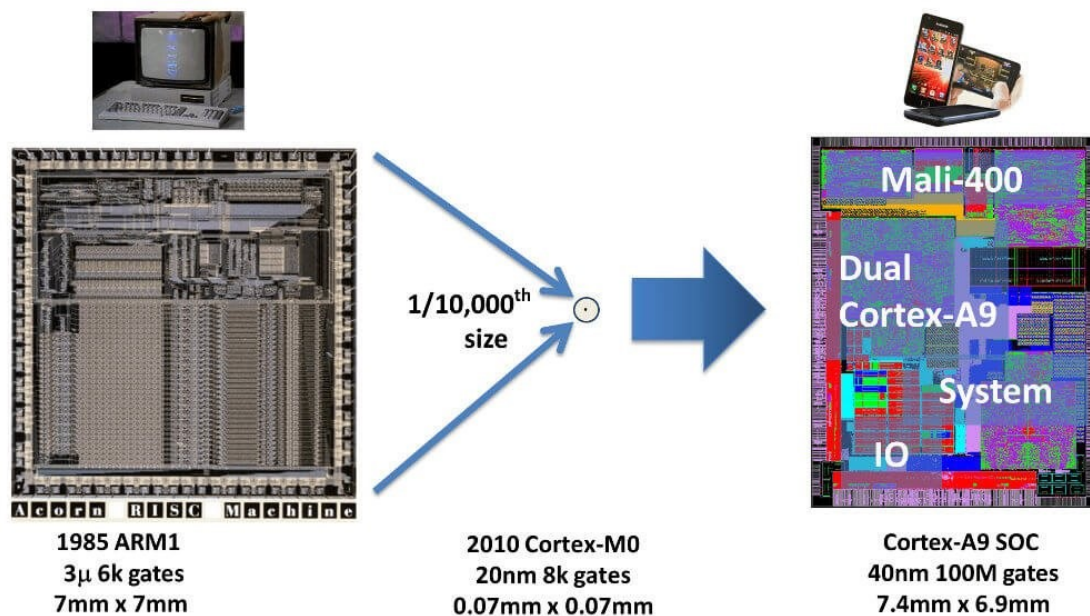


ARM History

- ARM Holdings (founded 1990), UK-based, Cambridge
- Founded as a joint venture between Acorn Computers, Apple Computer and VLSI Technology
- Inspired by papers from the Berkeley RISC project
- Originally: Acorn RISC Machine -> Advanced RISC Machines
 - 1980s: Co-processors for BBC Micro
 - 1985: ARM1 (replacing original 6502 design)
 - 1994: First ARM-powered phone Nokia6110 (TI chip)
 - 1997: ARM7TDMI (Thumb ISA combining advantages of CISC and RISC)
 - 2001: ARM926EJ-S (third-party licensing & HW-support for Java acceleration)
 - 2008: Cortex-A9 MPCore (smartphone market & multi-core processor introduction)
 - 2011: Introduction of the heterogeneous „big.LITTLE“ approach



ARM History ctd.



ARM History ctd.

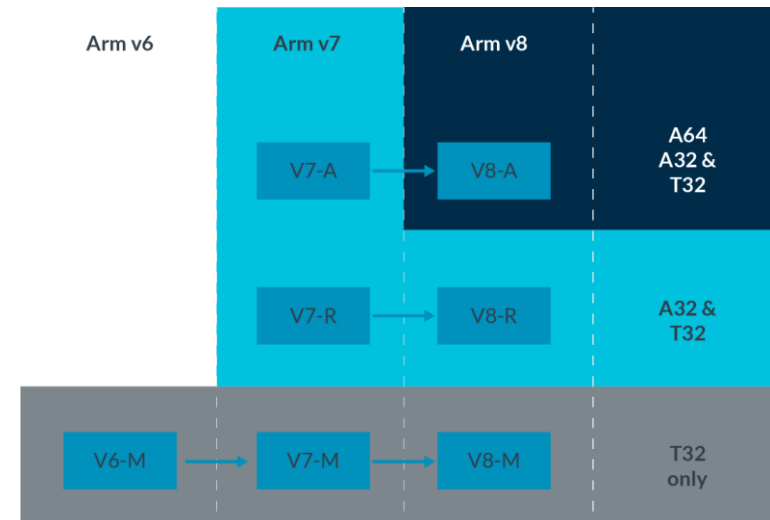
- ARM advantages: cost, energy efficiency, ...
- Why? -> RISC (reduced instruction set computing) architecture
 - Typically requires fewer transistors than CISC architecture (e.g. x86)
 - Improves cost, power consumption and heat dissipation
- Primary business: design of ARM processors (CPUs)
 - but also SW development tools, e.g. DS-5
 - and additional HW, e.g. SoC design, GPU (Mali) or debug/trace tools like DSTREAM
- Fabless company: selling IP cores (develop the architecture & license to others)
 - Core license: combine existing ARM core with other parts to produce a complete device
 - Architectural license: design own CPU cores using the ARM instruction set
 - Based on processor IP in synthesizable RTL, e.g. Verilog
 - Perform architectural optimisations & extensions (e.g. Apple, Qualcomm, ...)

ARM Portfolio

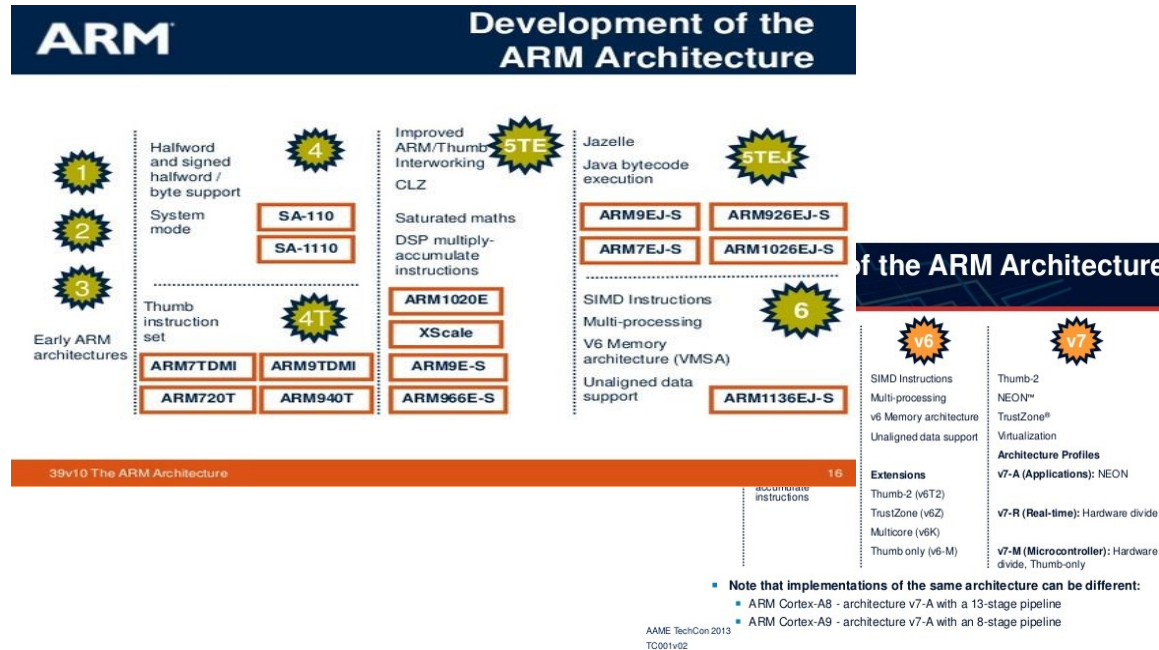
- CPU architecture: microarchitecture (e.g. processor power, pipeline length, cache), basic instruction set, exception & memory models (for OS, hypervisor)
 - A-Profile (Applications)
 - R-Profile (Real-Time)
 - M-Profile (Microcontroller)
- Processors
 - Cortex-A (High Performance/Rich Applications, e.g. Smartphone)
 - Cortex-R (Real-Time Processing/Determinism, e.g. Automotive)
 - Cortex-M (Energy Efficiency/Size, e.g. IoT)
 - Classic Processors (legacy processors – Arm 7, Arm 9, Arm 11)
- Platform Design (Server, Embedded, Machine Learning)
- System Architectures & Components (e.g. AMBA, CoreSight, ...)
- Graphics & Multimedia (Mali GPUs, video processors, etc.)

Modern ARM Architecture

- Armv8-A
 - A64, A32 and T32 instruction sets
 - 64-bit and 32-bit execution states, known as AArch64 and AArch32
 - > 4GB of physical memory
 - HW-accelerated cryptography
 - Backwards compatibility with Armv7-A architecture
 - VFPv3/v4 & advanced SIMD (NEON)
 - Ongoing development: ARMv8.X-A extensions
- Armv7-A
 - A32 and T32 instruction sets
 - Virtualization & multiprocessing extensions
 - Security extensions (TrustZone)



ARM Architecture Evolution

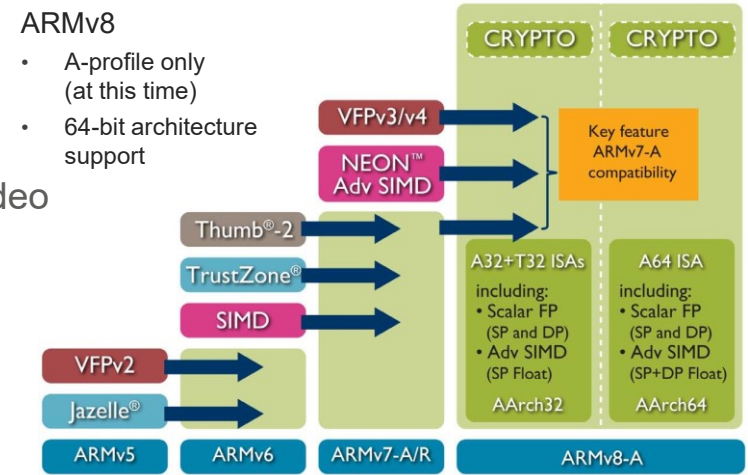


Excursus: Instruction Set Architecture (ISA)

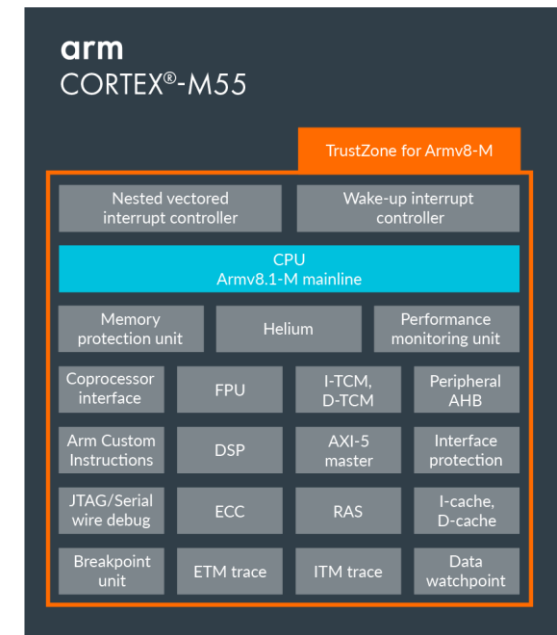
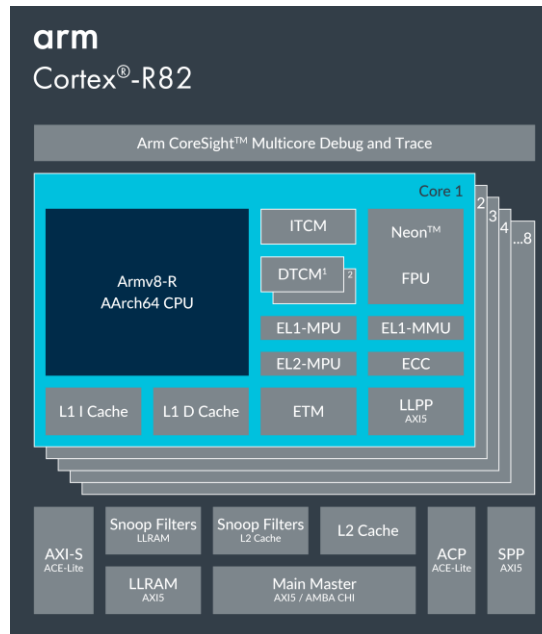
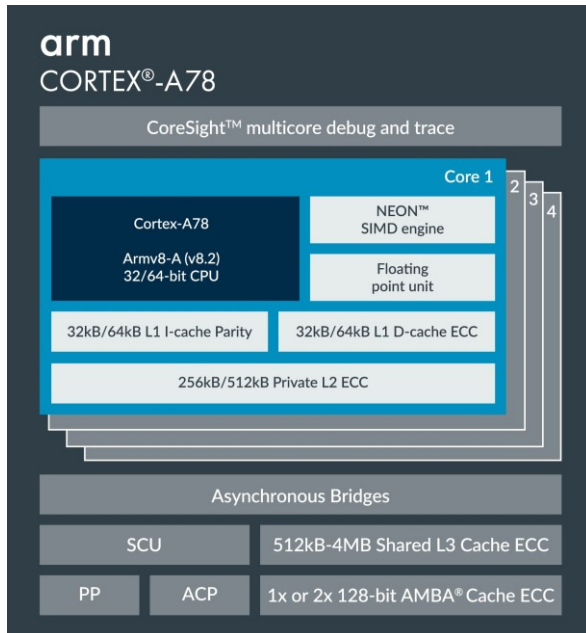
- Abstract model of a computer
- ISA implementations can vary in performance, physical size, and monetary cost
- Interface between SW and HW (binary compatibility)
- Definition of
 - The supported data types
 - The state (e.g. main memory and registers)
 - Their semantics (e.g. memory consistency, addressing modes)
 - The instruction set (set of machine instructions that comprises a computer's machine language)
 - And the input/output model
- Defines everything a machine language programmer needs to know in order to program a computer
- Extensions ?!

ARM Instruction sets + extensions

- Load/store architecture: instructions divided in memory access & ALU operations (vs. register-memory architecture)
 - Different CPU modes (e.g. hyp, user, ...)
 - Different registers (multi-purpose & special purpose, e.g. SP, PC)
 - Integer arithmetic operations: add, subtract, multiply (and divide)
- ARM (A32/A64)
- Thumb/Thumb-2 (16/32-bit): compact subset of ARM instruction set
- Jazelle: Java bytecode execution
- Extensions
 - NEON/Advanced SIMD: signal processing, e.g. audio/video
 - VFP (Vector Floating Point): FPU coprocessor
 - DSP (Digital Signal Processing)
 - Security Extensions (e.g. TrustZone)

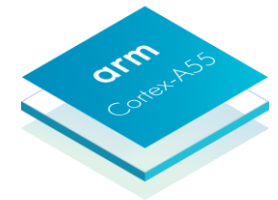


ARM Cortex Architecture



ARM Cortex-A

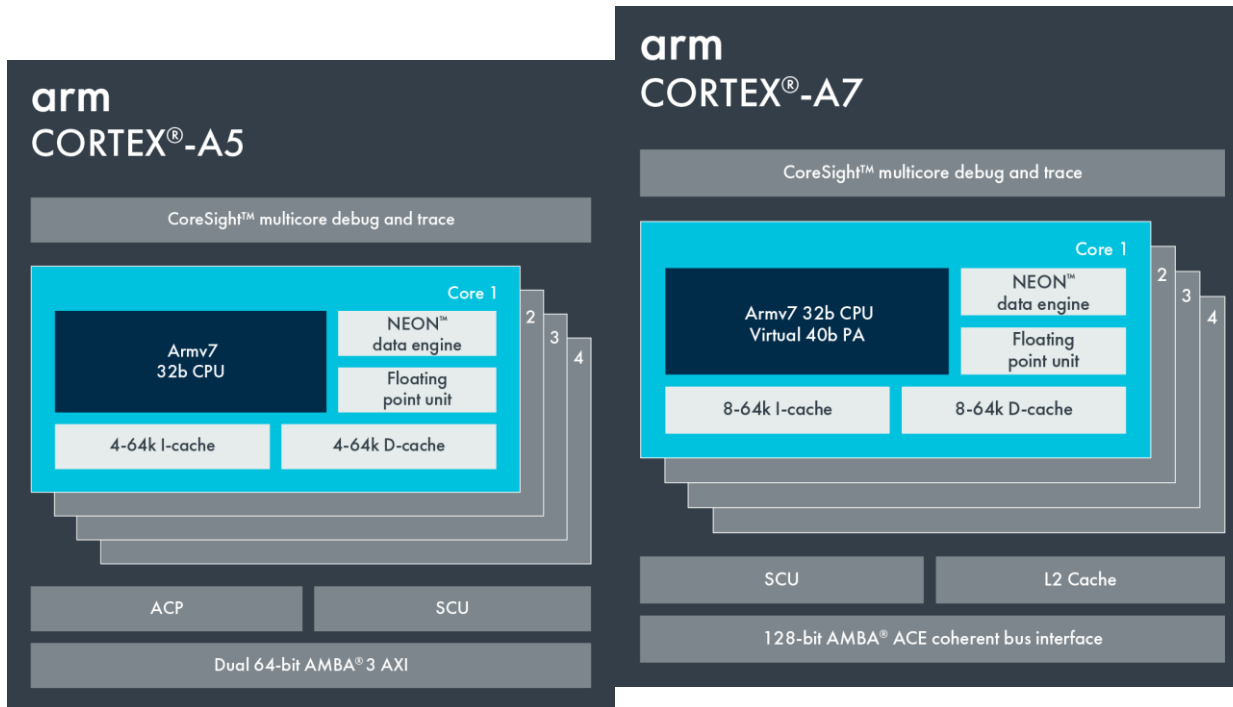
- ARMv8.2-A (64-bit)
 - 2020: Cortex-X1
 - 2020: Cortex-A78(C), Cortex-A78AE
 - 2019: Cortex-A77
 - 2018: Cortex-A76(AE), Cortex-A65AE
 - 2017: Cortex-A75, Cortex-A55
- ARMv8-A (64-bit)
 - 2019: Cortex-A34
 - 2016: Cortex-A73
 - 2015: Cortex-A72, Cortex-A35
 - 2012: Cortex-A57, Cortex-A53
- ARMv8-A (32-bit)
 - 2016: Cortex-A32
- ARMv7-A (32-bit)
 - 2014: Cortex-A17
 - 2013: Cortex-A12
 - 2011: Cortex-A7
 - 2010: Cortex-A15
 - 2009: Cortex-A5
 - 2007: Cortex-A9
 - 2005: Cortex-A8
- Third-party
 - Apple Ax
 - Nvidia Tegra
 - Qualcomm Snapdragon
 - ...



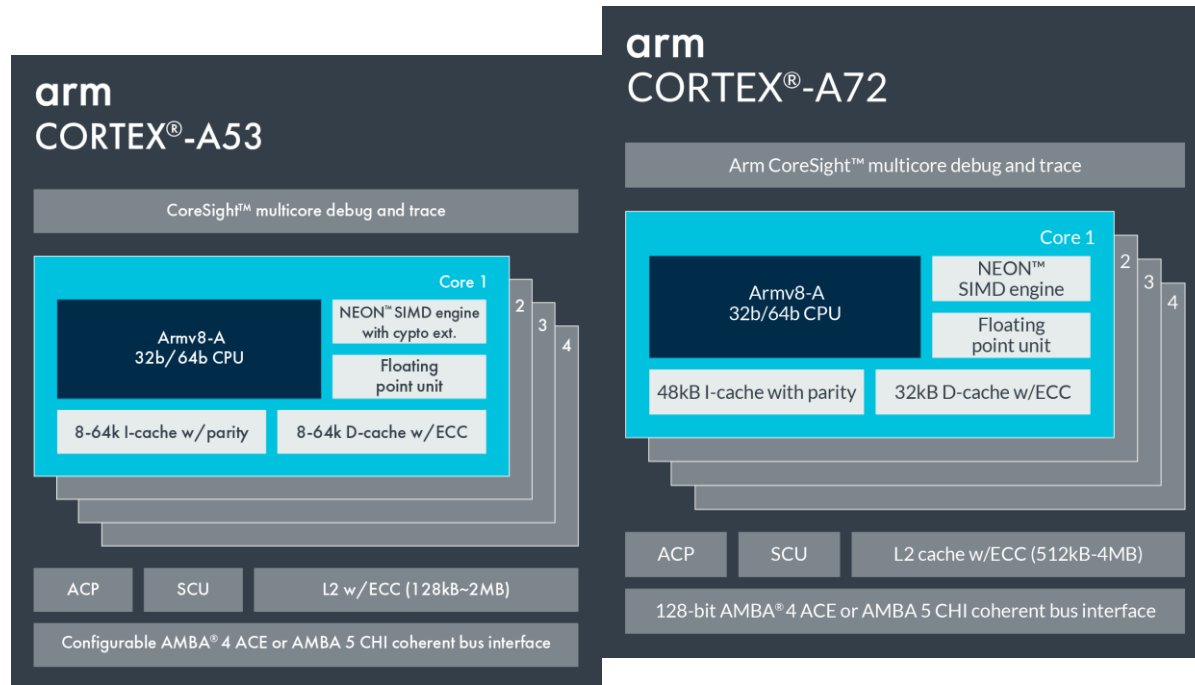
ARM Cortex-A ctd.

- 32-bit/64-bit RISC ARM processor
- Intended for Rich Applications or OS usage
- Implementation of ARM Application (A) profile
- Instruction set
 - ARMv8-A: ARM (64-bit), ARM (32-bit), Thumb (32-bit)
 - ARMv7-A: ARM (32-bit), Thumb-1 (16-bit), Thumb-2 (32-bit), Jazelle (8-bit Java bytecode)
- Targeting laptop-class performance with Cortex-A76 (+ Cortex-A55 in scalable DynamIQ big.LITTLE configuration)
- Extending focus on automotive market with Cortex-A65AE, Cortex-A76AE and Cortex-A78AE: safety, mixed-criticality, ...

ARMv7-A: Cortex-A5 & Cortex-A7



ARMv8-A: Cortex-A53 & Cortex-A72



ARM Roadmap

Delivering Stunning Results in all Markets



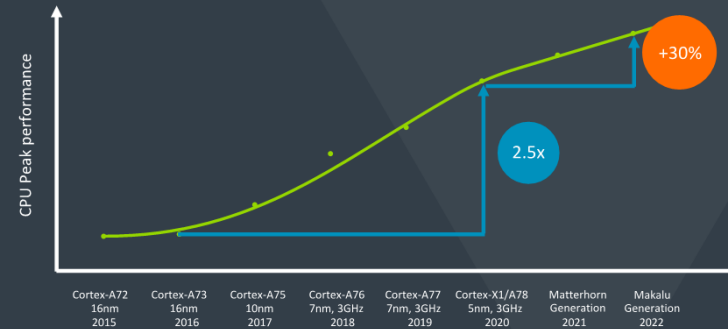
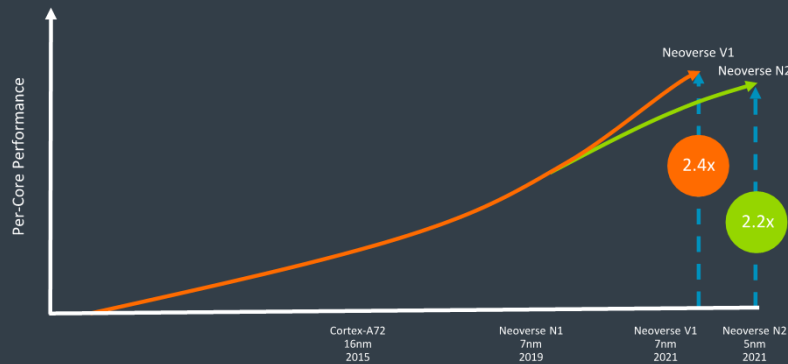
Arm Neoverse V1 platform:
Arm's fastest per-core performance for demanding HPC, cloud, and machine learning workloads



Arm Neoverse N2 platform:
High per-core performance and efficiency enable the highest scale out performance in hyperscale, 5G edge, and SmartNIC

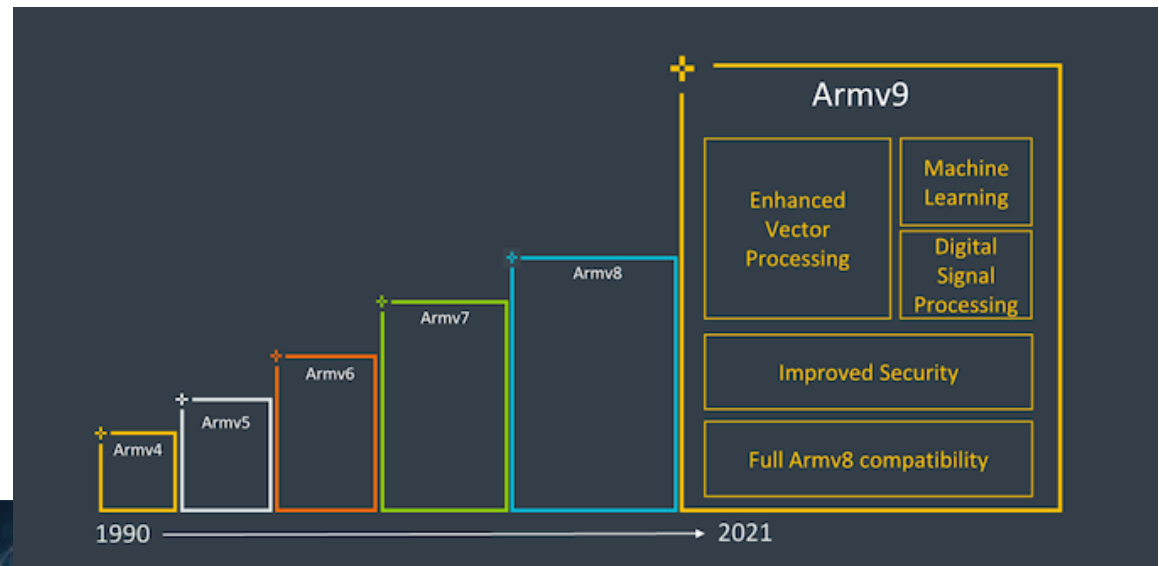


Cortex-X:
Delivers ultimate peak performance within an expanded envelope for power and area



Measured estimates on SPECint*_rate2006 (SPECspeed* Integer component of SPEC CPU*_rate2006), Arm single-core performance estimated for compute platform. Results are measured estimates using specific computer systems, software, components, operations, and functions and changes to any of these factors will cause the results to vary. Graph not to scale.

ARMv9



Foundations of the Armv9 Decade



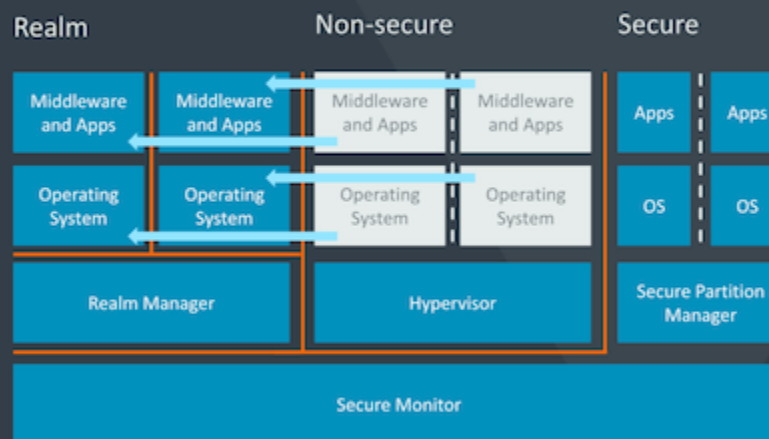
ARMv9 ctd.

The Arm Confidential Compute Architecture

Arm CCA

- **Realms** enable you to run an application or service in such a way that your data is protected from:

- The host
- The hardware manufacturer or service supplier
- Other software running on the device or sharing the machine



ARM Cortex-R

- ARMv8-R (64-bit)
 - 2020: Cortex-R82 (optional MMU, Linux capable)
- ARMv8-R (32-bit)
 - 2016: Cortex-R52(F)
- ARMv7-R (32-bit)
 - 2016: Cortex-R8(F)
 - 2011: Cortex-R7(F), Cortex-R5(F), Cortex-R4(F)

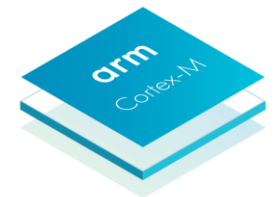


ARM Cortex-R ctd.

- 32-bit/64-bit RISC ARM processor
- Optimized for hard real-time and safety-critical applications
- Safety/fault tolerance features: e.g. MPU, ECC (L1 cache/buses), dual-core lockstep (CPU fault tolerance), deterministic interrupt handling
- Implementation of ARM Real-Time (R) profile
- Instruction set
 - ARMv7-R/ARMv8-R
 - ARM (32-bit/64-bit)
 - Thumb-1 (16-bit), Thumb-2 (32-bit)

ARM Cortex-M

- ARMv8.1-M Mainline (32-bit, Harvard) [entire Thumb-1/2 + TZ + ML capabilities]
 - 2020: Cortex-M55
- ARMv8-M Mainline (32-bit, Harvard) [entire Thumb-1/2 + TZ]
 - 2018: Cortex-M35P(F), 2016: Cortex-M33(F) [cross of Cortex-M4 & Cortex-M23]
- ARMv8-M Baseline (32-bit, von-Neumann) [most Thumb-1, some Thumb-2]
 - 2016: Cortex-M23 [Cortex-M0+ + TZ]
- ARMv7E-M (32-bit, Harvard) [entire Thumb-1/2]
 - 2014: Cortex-M7(F) [Cortex-M4 + VFPv5 (Sp+DP FPU)]
 - 2010: Cortex-M4(F) [Cortex-M3 + DSP + VFPv4-SP (SP FPU)]
- ARMv7-M (32-bit, Harvard) [entire Thumb-1/2]
 - 2004: Cortex-M3
- ARMv6-M (32-bit, von-Neumann) [most Thumb-1, some Thumb-2]
 - 2012: Cortex-M0+, 2009: Cortex-M0, 2007: Cortex-M1 (focus on FPGA implementation)

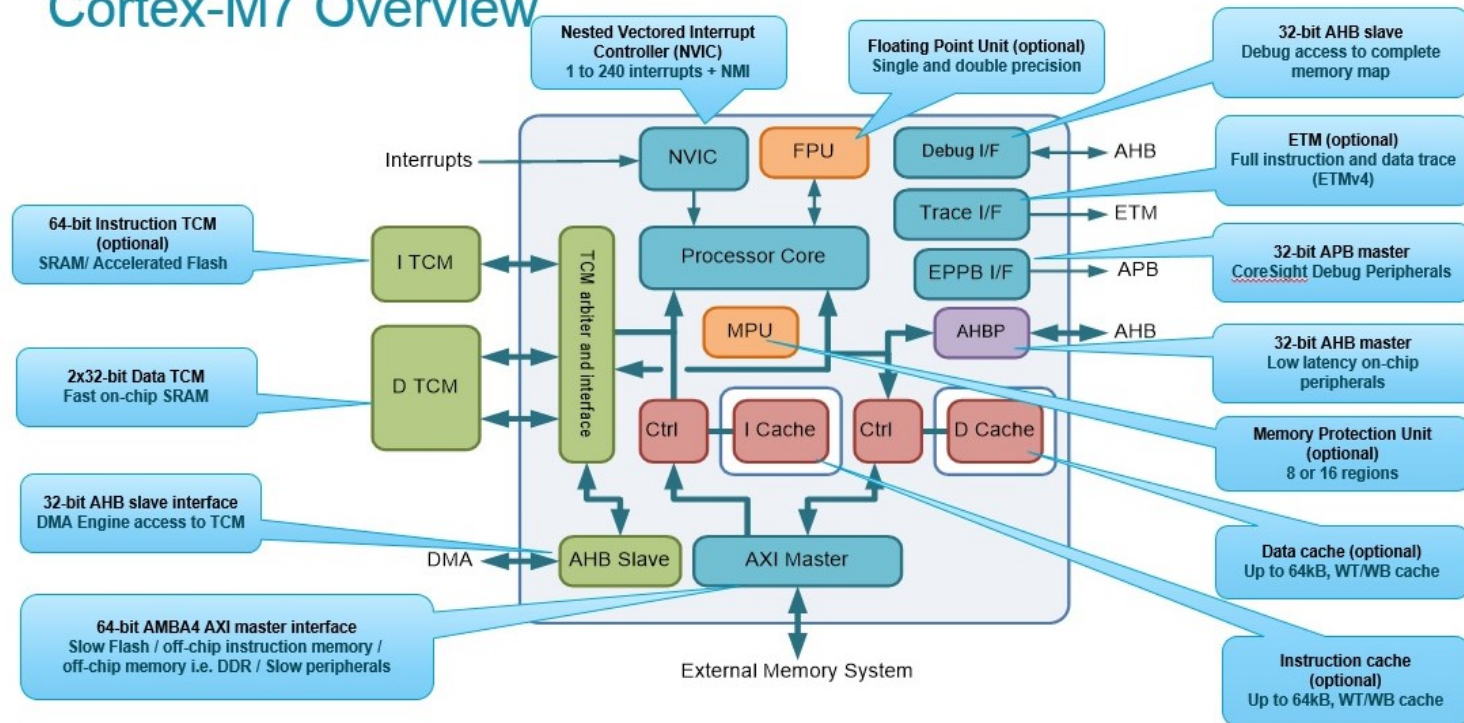


ARM Cortex-M ctd.

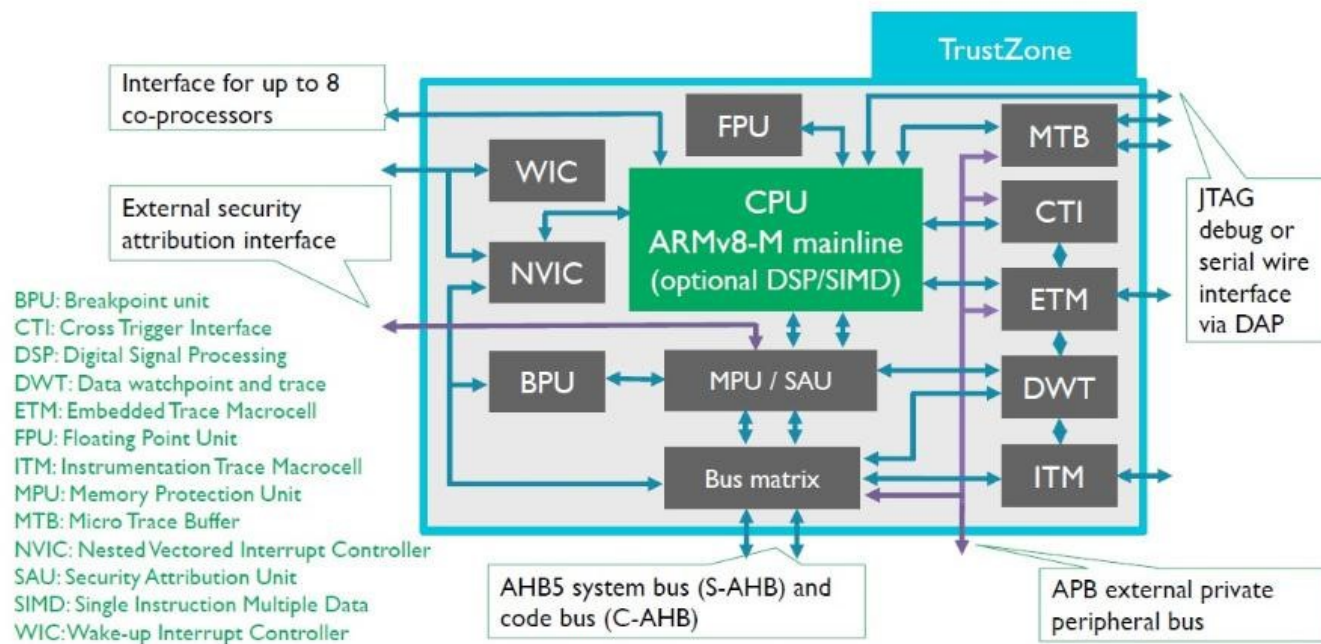
- 32-bit RISC ARM processor
- Intended for microcontroller use
- Implementation of ARM Microcontroller (M) profile
- FPU silicon option (Cortex-MxF)
- Instruction set
 - No ARM (32-bit)
 - Most Thumb-1 (16-bit), some Thumb-2 (32bit)
- Binary compatibility
 - Cortex-M0/M0+/M1 -> Cortex-M3/M4/M7
 - Cortex-M3 -> Cortex-M4/M7/M33/M35P

Example: ARM Cortex-M7

Cortex-M7 Overview



Example: ARM Cortex-M33



ARM Cortex-M - Evolution

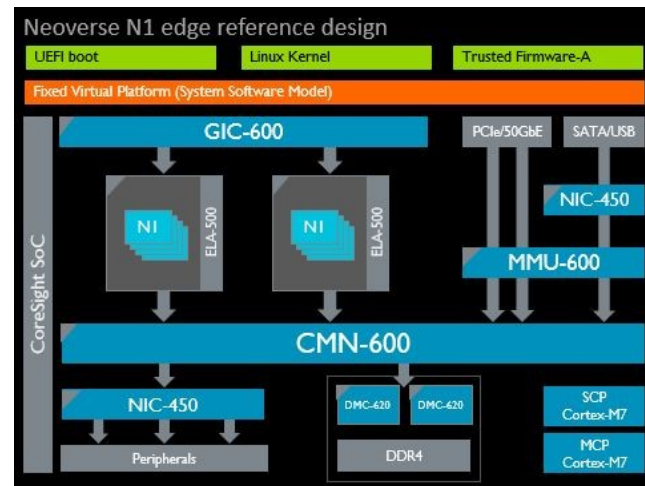
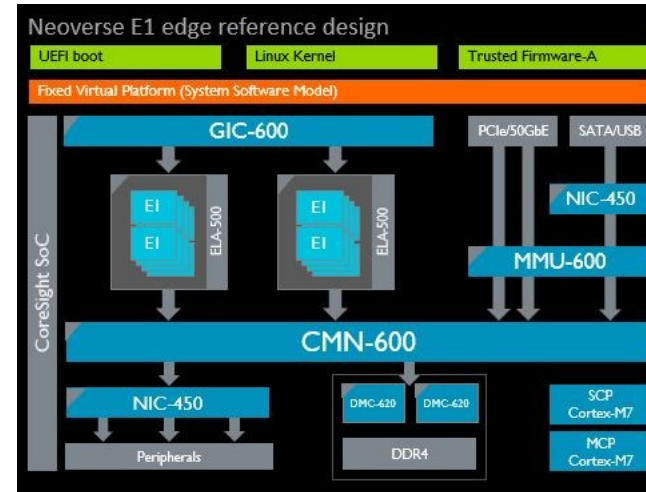
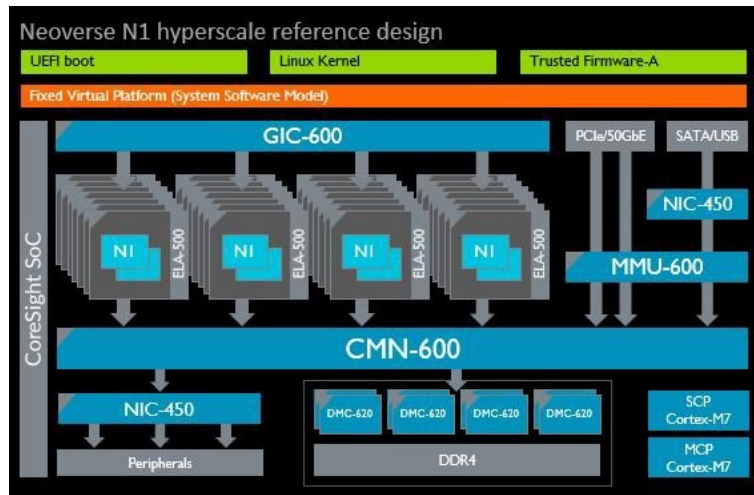
Cortex-M4		Cortex-M33	
	ETM		TrustZone
	NVIC (max 240 IRQs)		Stack limit checking
	MPU (PMSAv7)		Co-processor interface
	AHB Lite		Enhanced debug
	FPU		MTB
	SIMD/ DSP		ETM
	WIC		NVIC (max 480 IRQs)
	Serial wire / JTAG		MPU (PMSAv8)
	ARMv7-M		AHB5
			FPU
			SIMD/ DSP
			WIC
			Serial wire / JTAG
			ARMv8-M mainline

■ New or updated

Excursus: ARM Server & Infrastructure

- As of 2014: started with designs like Cavium ThunderX
 - re-using ARM Cortex-A designs & transitioning to server area
 - Mixed results -> mostly not competitive with x86-based Intel and AMD
- As of 2019: ARM introduced dedicated design with ARM Neoverse N1/E1
 - Supports the ARMv8.2-A architecture and AArch64 ISA
 - E1 Edge: 16x Neoverse E1 CPUs – compute intensive edge cloud, network, storage and security
 - N1 Hyperscale: 64x-128x Neoverse N1 CPUs – highest performance server and cloud applications
 - N1 Edge: 8x Neoverse N1 CPUs – compute intensive edge cloud, network, storage and security
 - Example: Amazon AWS Graviton2 processor instances
- Third-party designs (ARMv8-based)
 - Cavium ThunderX2, Marvell ThunderX3
 - Fujitsu A64FX (using ARM SVE)
 - Huawei Kunpeng 920 (64 cores @ 2.6 GHz)

Excursus: ARM Server & Infrastructure

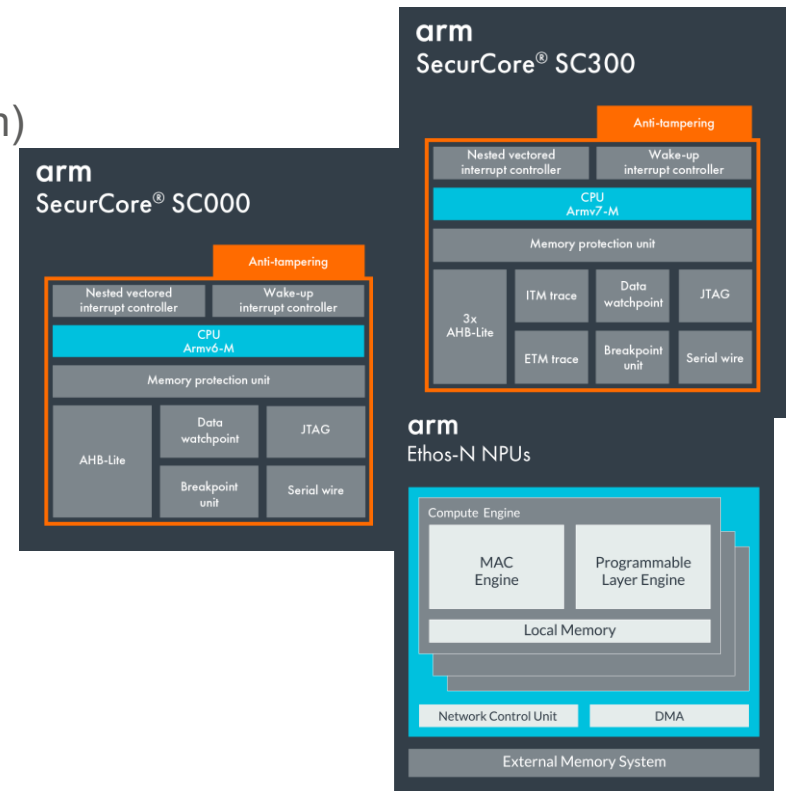


ARM Cortex - Documentation

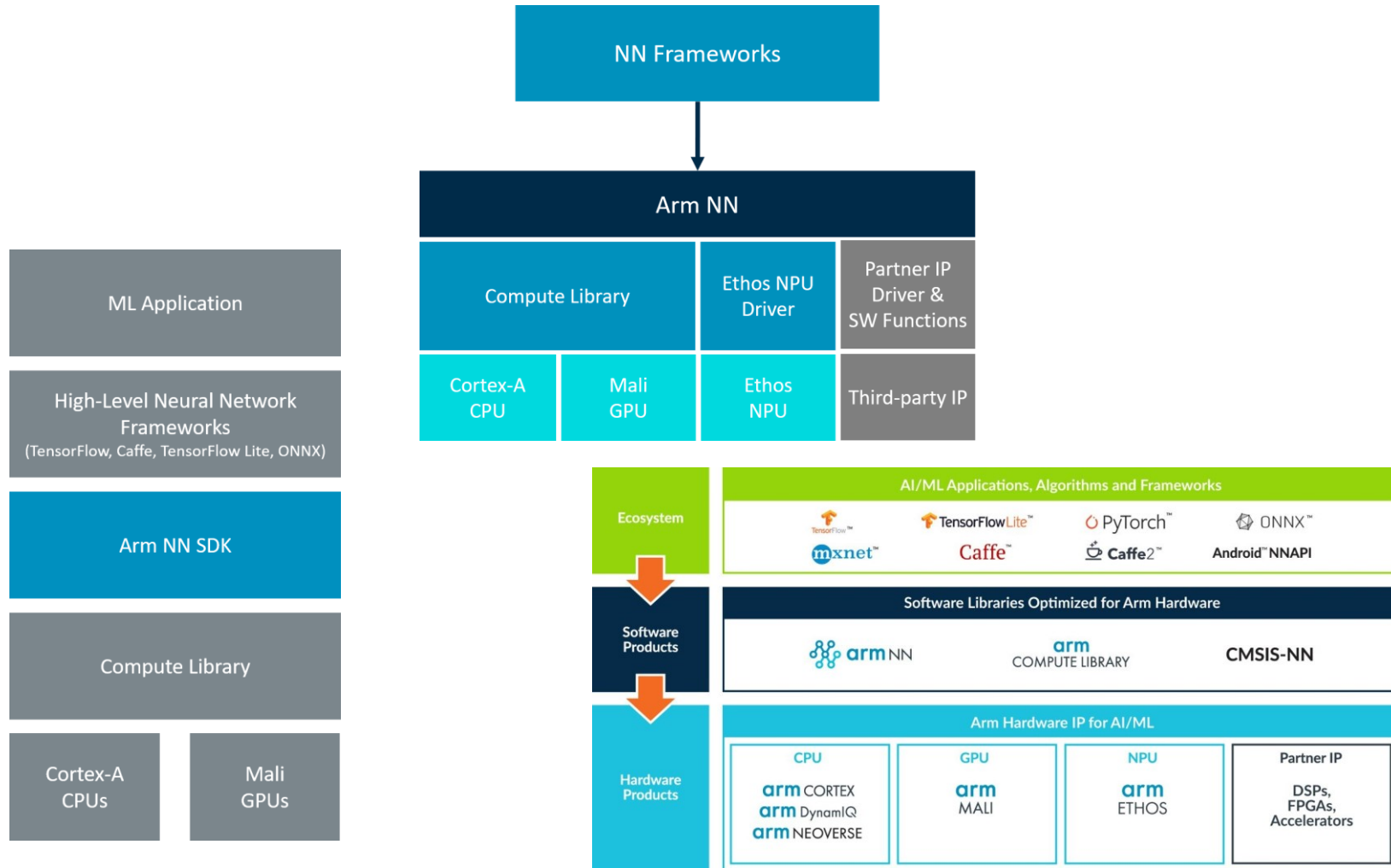
- Typical documentation tree
 - IC manufacturer website, high-level marketing slides, datasheet (for the exact physical chip) and reference manual (for common peripherals and other aspects of chip family)
 - ARM core website, generic user guide, technical reference manual (describes the ISA) and architecture reference manual (detailed description of all instruction sets of the core)
- IC manufacturers provide additional documents
 - Evaluation board user manuals
 - Application notes
 - Getting started guides
 - Software library documents, errata and more

ARM – Specific processors

- SecurCore (e.g. smartcards, payment systems)
 - SC300 (Cortex-M3 + enhanced security)
 - SC000 (Cortex-M0 + enhanced security)
- Machine Learning Accelerators (ex. Project Trillium)
 - ARM Ethos-N (cloud)
 - Ethos-N78
 - Ethos-N77
 - Ethos-N57
 - Ethos-N37
 - ARM Ethos-U (embedded)
 - Ethos-U65
 - Ethos-U55

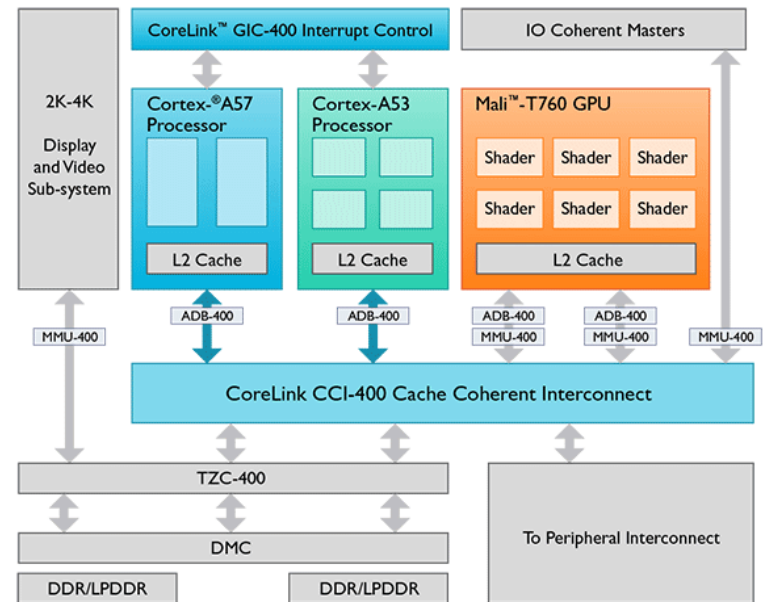


Excursus: ARM Machine Learning



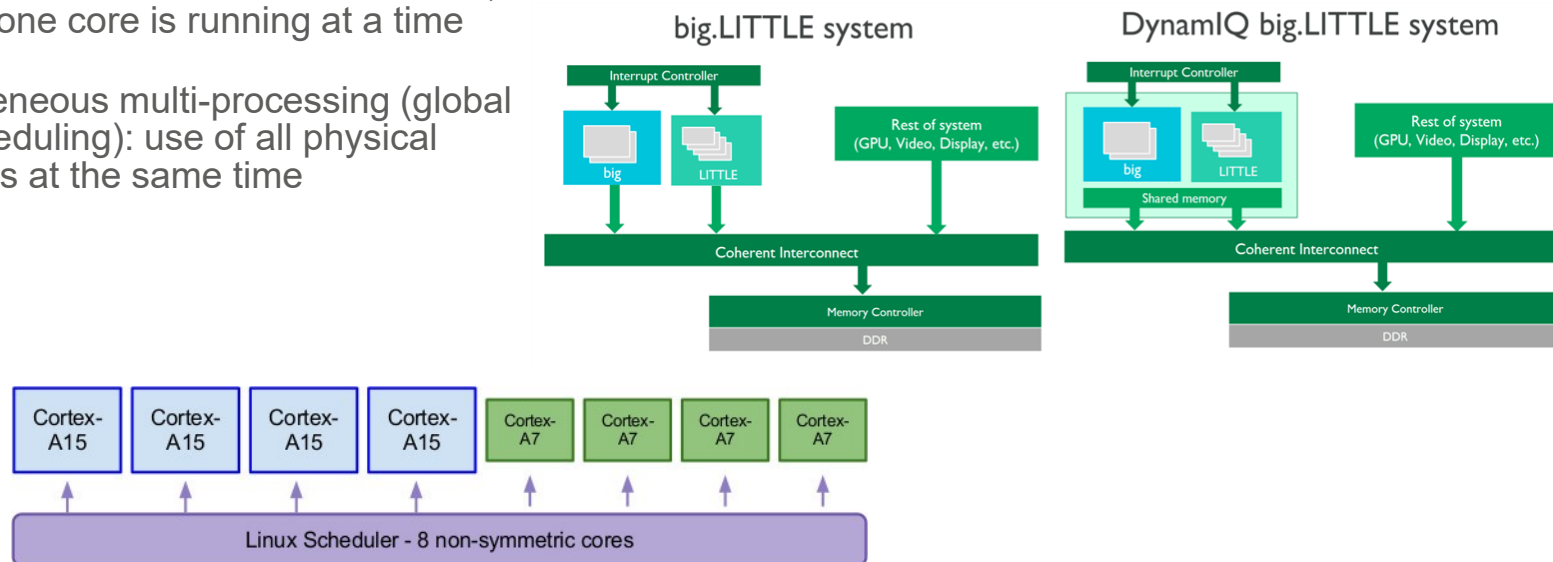
ARM big.LITTLE

- ARM big.LITTLE
 - Heterogeneous processing architecture that uses two types of processors
 - Cluster of high-performance (big) & high-efficiency (LITTLE) CPUs
 - Software layer (Global Task Scheduling): map the right application task onto the right CPU
 - Both types of processors are coherent and share the same ISA
- Successor: ARM DynamIQ (combining the big and LITTLE CPUs into a single, fully-integrated cluster)



ARM big.LITTLE ctd.

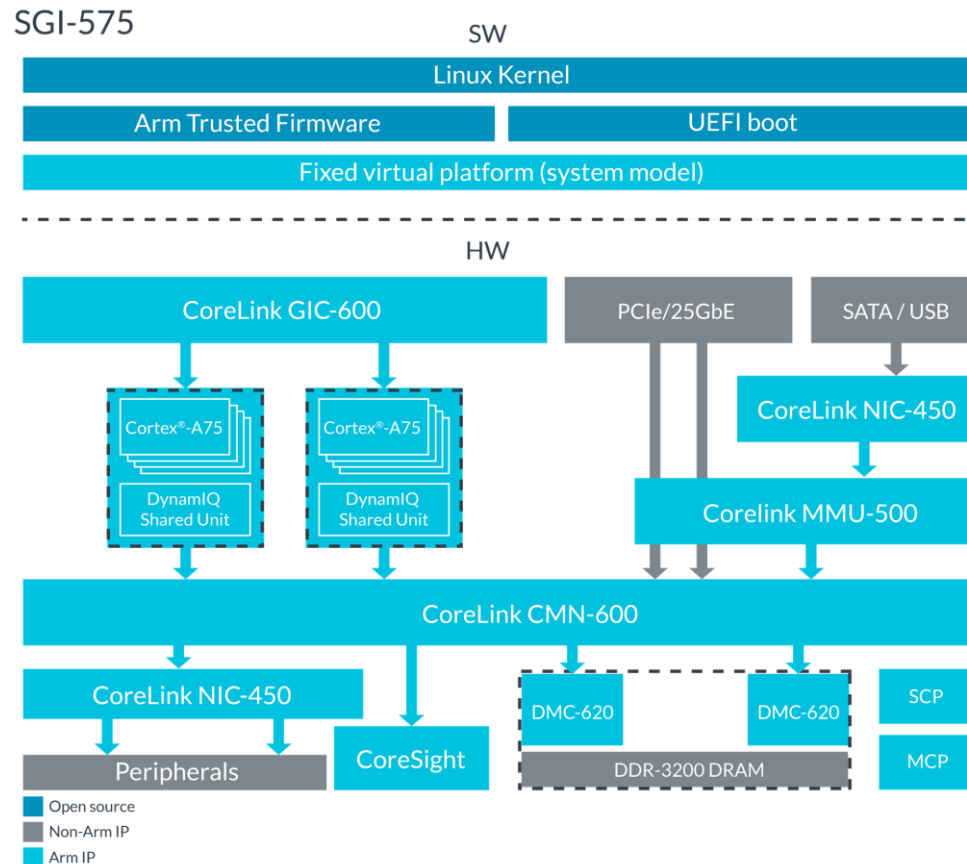
- Usage modes
 - Clustered switching: processor is arranged into identically-sized clusters of big and LITTLE cores, but only one cluster is used at a time
 - In-kernel switcher (CPU migration): pairing a big and LITTLE core to a virtual core, but only one core is running at a time
 - Heterogeneous multi-processing (global task scheduling): use of all physical resources at the same time



Example: ARM SoC structure & components

- Mali Multimedia IP
- System IP (built on AMBA interface)
 - Interconnects
 - Security IP
 - System Controllers
 - Debug and trace
- CoreLink Memory Controllers
- CoreLink Interconnect
 - CoreLink Coherent Mesh Network (CMN)
 - CoreLink Cache Coherent network Family (CCN)
 - CoreLink Cache Coherent Interconnect (CCI)
 - CoreLink Network Interconnect (NIC)
- System Controllers
 - System MMU
 - CoreLink GIC
 - Other System Controllers (e.g. DMA)
- Security
 - Cryptosland & CryptoCell family
 - TrustZone
- CoreSight Debug and Trace
 - CoreSight ETM
 - CoreSight PTM
 - CoreSight STM
 - CoreSight TMC
 - CoreSight ELA

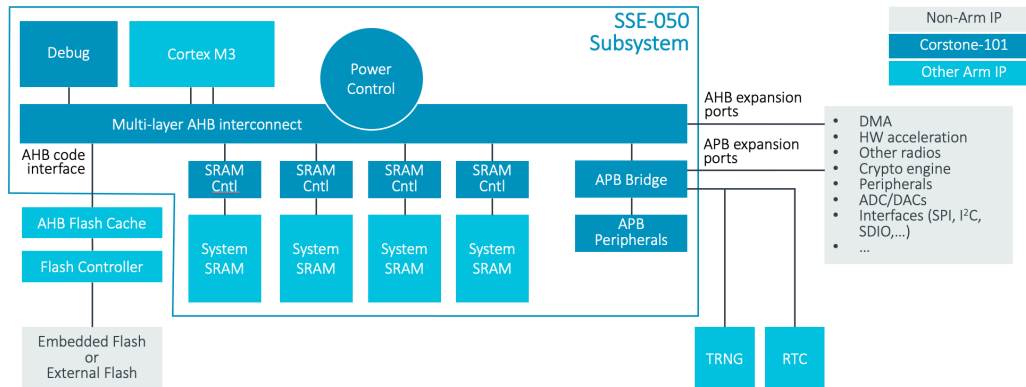
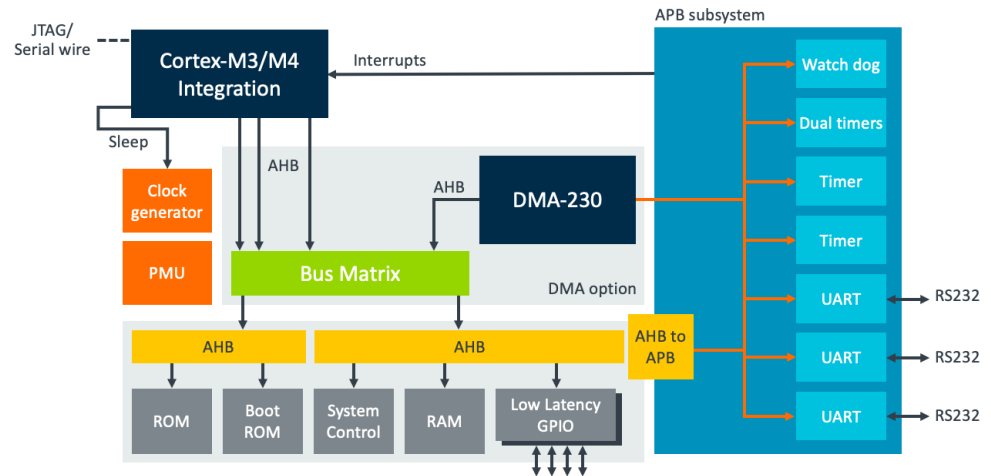
Example: ARM Interconnects



AMBA

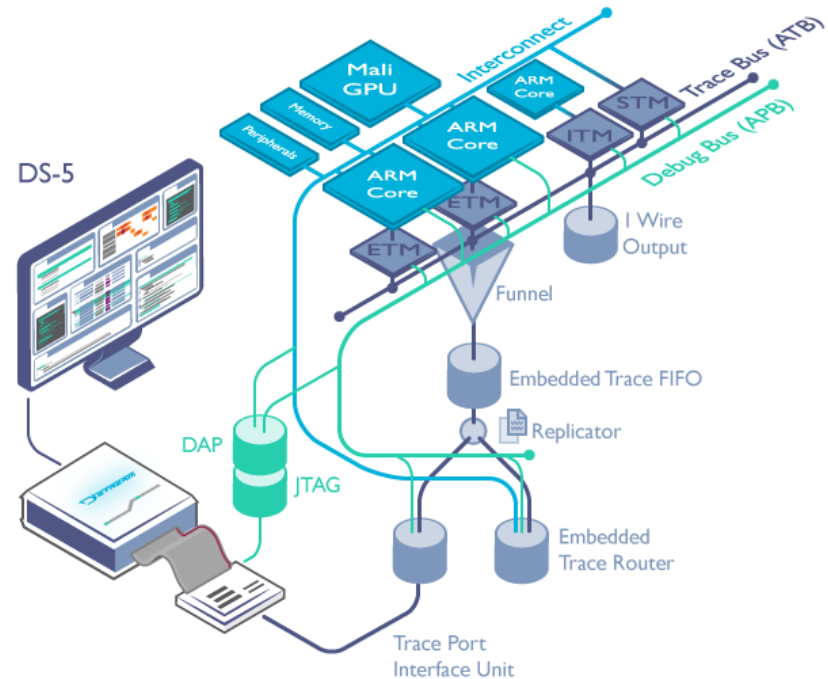
- The Arm AMBA (Advanced Microcontroller Bus Architecture) protocols are
 - An open standard, on-chip interconnect specification
 - for the connection and management of functional blocks in a SoC
- Types
 - GFB (Generic Flash Bus Controller)
 - CHI (Coherent Hub Interface)
 - AHB (Advanced High-Performance Bus): main system bus in microcontroller usage
 - AXI (Advanced eXtensible Interface): most widespread AMBA interface
 - ACE (AXI Coherency Extensions): used in big.LITTLE systems (e.g. smartphones)
 - APB (Advanced Peripheral Bus)
 - ATB (Advanced Trace Bus): CoreSight Debug and Trace
 - LPI (Low Power Interfaces)

AMBA

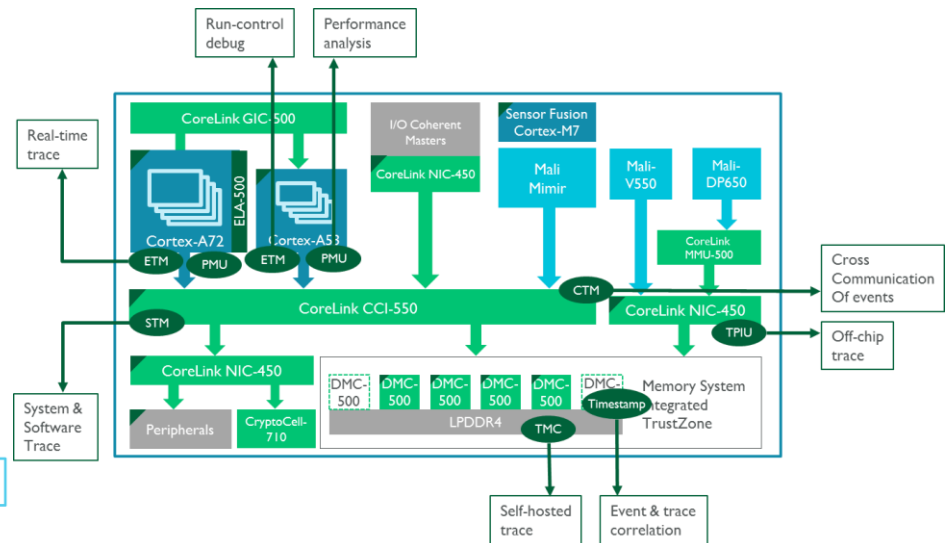
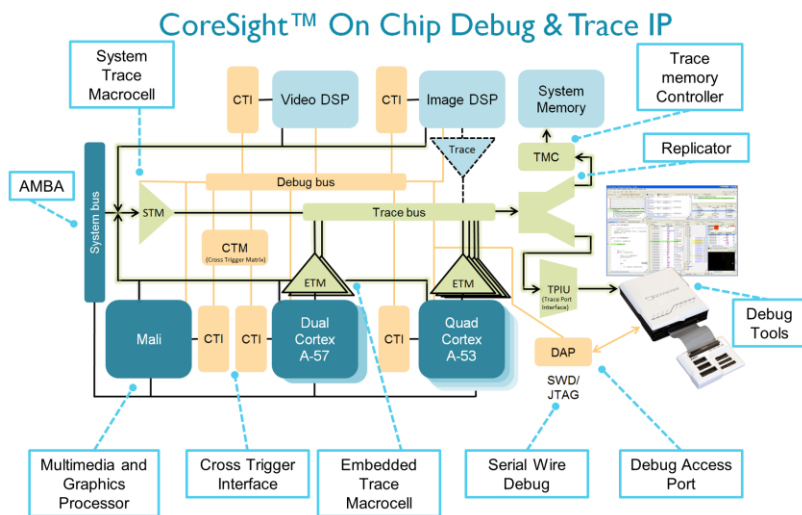


ARM Debug & Trace

- ULINK (Cortex-M, ARM7/9 & Keil MDK)
 - ULINK2 (SWD, SWV, JTAG)
 - ULINKplus (+ multi-core debugging)
 - ULINKpro (+ ETM trace)
- DSTREAM (ARMv4-8, 4GB trace buffer & DS-5)
 - Instruction Trace Analysis: ETM & PTM
- DSTREAM-ST (Supported protocols include STM, ITM, ETB, ETM and PTM)
- HSSTP (High Speed Serial Trace Probe)



ARM CoreSight – SoC Components



ARM Development - Simulation

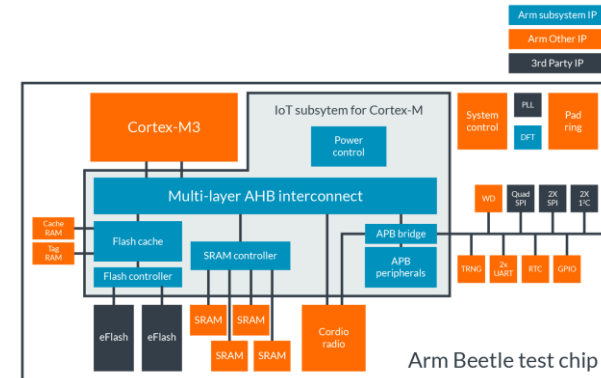
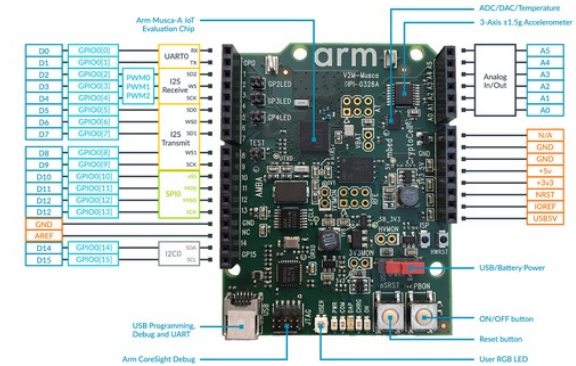
- Cycle Models (100% functional & cycle accuracy; compiled directly from RTL)
 - Both CPU (e.g. Cortex-A72) & System IP (e.g. Interrupt Controller) Cycle Models
 - Cycle Model Studio: compile RTL into high-speed software models
 - Used in: SoC Designer, SystemC based simulation environments and RTL simulation environments (Verilog/VHDL)
 - CPAKs (Cycle Performance Analysis Kits): ready-to-use virtual prototypes
- Fast Models (fast & functionally accurate programmer's view models)
 - CPU, System IP and Media IP Fast Models + peripherals and infrastructure (e.g. UART)
 - SW development before HW is available (low-level SW development, e.g. OS kernels)
 - Exportable to SystemC and TLM 2.0
- Fixed Virtual Platforms (built with ARM fast Models)
 - Speed comparable to real HW
 - Complete simulation of an ARM system (including processor, memory and peripherals)

ARM Development – Virtual Prototype

- A complete virtual prototype of a system contains more than just an Instruction Set Simulator.
- A full system consist of:
 - Fast, accurate models of cores, subsystems or systems
 - SystemC interface for integration with EDA tools and other IP blocks
 - APIs for debug (CADI) and trace (MIT), allowing full control and an interface to DS-5, MDK and 3rd party debug tools
 - Python based scripting for runtime control, checking and reporting
 - Visualization, file system access, peripherals and networking from virtual I/O
 - Fully compatible Linaro SW stacks, from boot code to Linux and Android OS support

ARM Development – Reference Boards

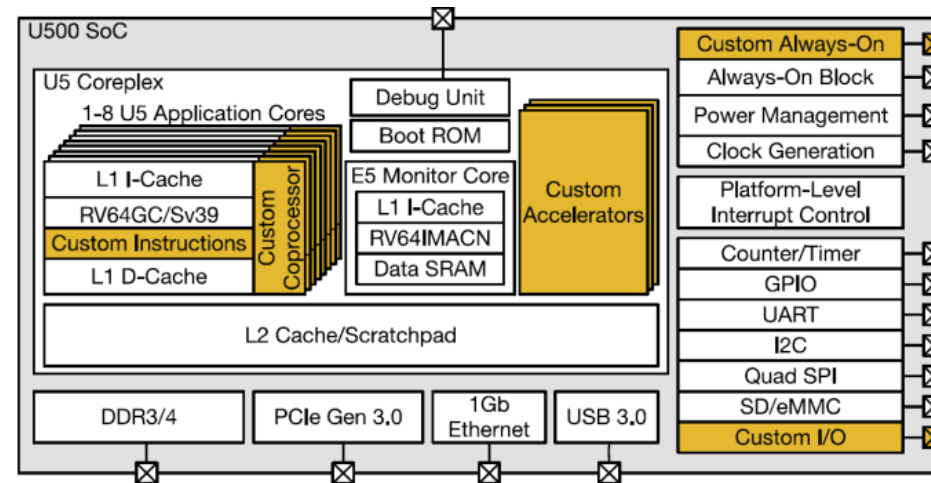
- Cortex-A (ARMv8-A)
 - Neoverse N1 System Development Platform
 - Juno Platform (Cortex-A72/Cortex-A53 big.LITTLE)
 - Versaile Express LogicTile (plug-in FPGA daughterboards)
- Cortex-R
 - Soft Macrocell Model (FPGA implementation of a Cortex-R5/R7)
- Cortex-M
 - Musca-B1 test Chip Board (Cortex-M33)
 - Musca-A Test Chip Board (2x Cortex-M33)
 - Beetle IoT Evaluation Board (Cortex-M3)
- FPGA
 - MPS2+ FPGA Prototyping Board (Cortex-M)
 - MPS3 FPGA Prototyping Board (up to small Cortex-A)



RISC-V

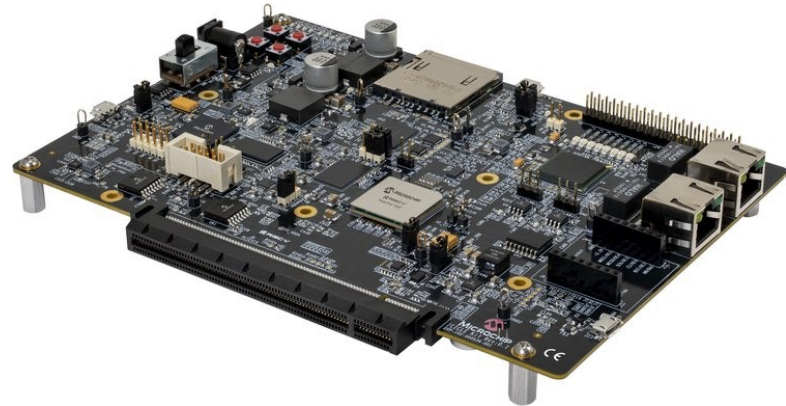
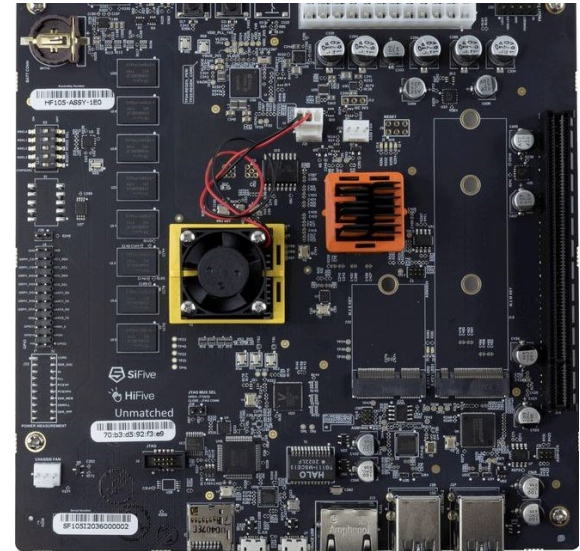
RISC-V

- Open ISA (RISC-based) from University of California, Berkeley
- Introduced 2010
- Can be freely used for any purpose, permitting anyone to design, manufacture and sell RISC-V chips and software
- Several open source designs:
 - UCB BAR: 64-bit BOOM, 64-bit Rocket, 32-bit Sodor, ...
 - Third-party: ETH/OpenHW Group CVA6, IIT Madras Shakti, ...
- Commercial designs: SiFive (Freedom E310, Freedom U540, ...)



RISC-V - Boards

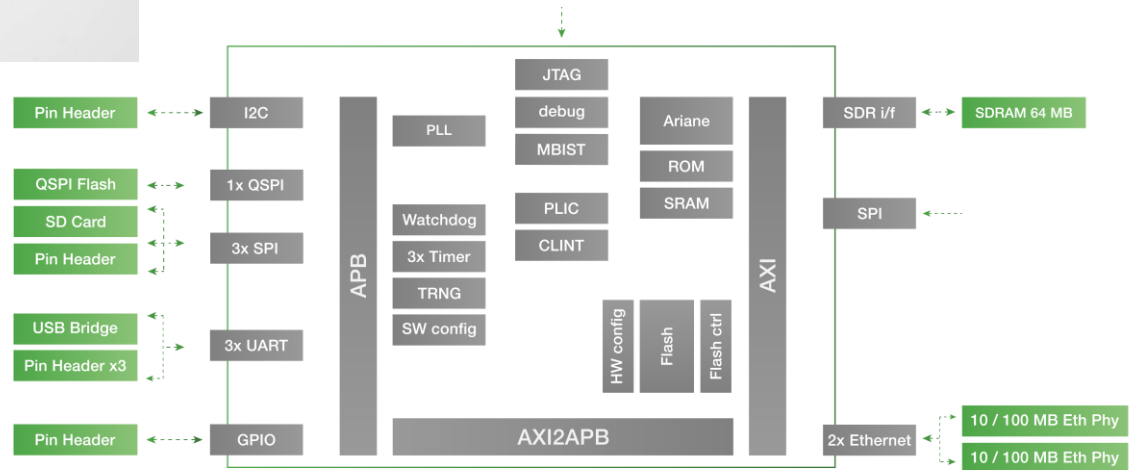
- Problem: mostly 32-bit platforms comparable to ARM Cortex-M
- Upcoming 64-bit, Linux-capable development platforms
 - SiFive HiFive Unmatched
 - Microchip PolarFire SoC Icicle Kit (including FPGA)
- Current problems
 - Affordable HW
 - Comparable performance to ARM
 - Availability of free IP for peripherals, buses, ...



HENSOLDT Cyber - MiG-V



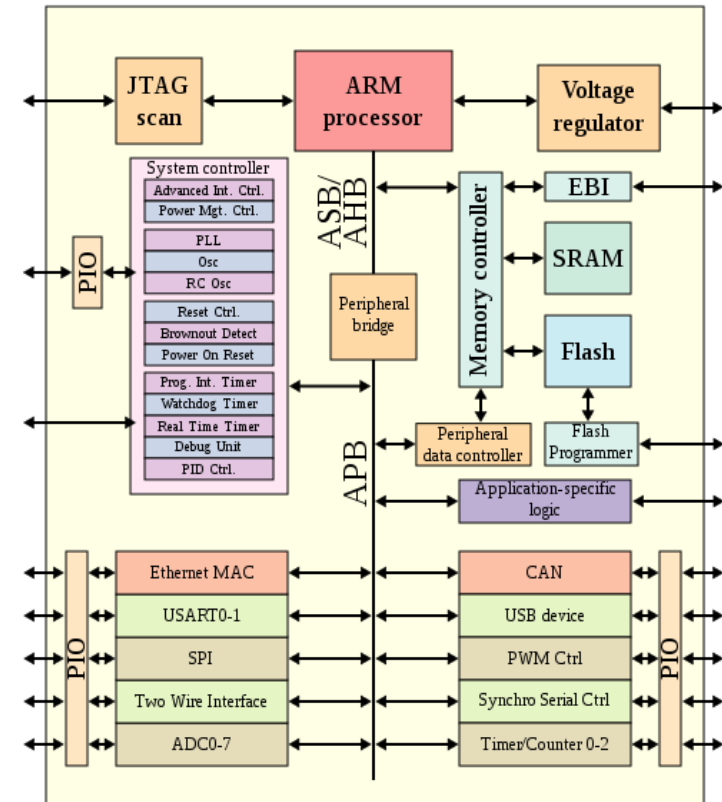
MiG-V



SoC

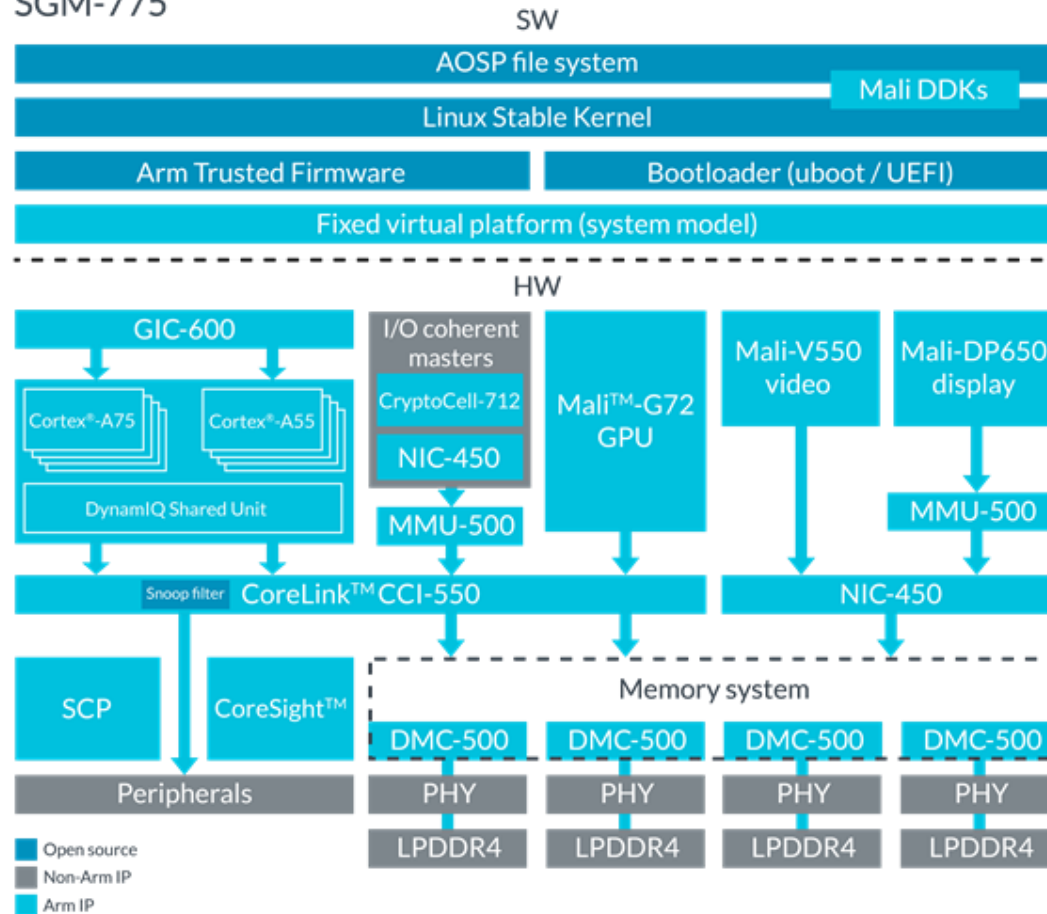
System on chip (SoC) - Definition

- A SoC is an integrated circuit that
 - integrates all components of a computer
 - on a single circuit die
- In contrast to traditional motherboard architecture (detachable/replaceable components/modules as discrete components/expansion cards)
- Replaces traditional microcontroller (application areas)
- Typically used in mobile computing
 - Smartphones
 - Embedded systems



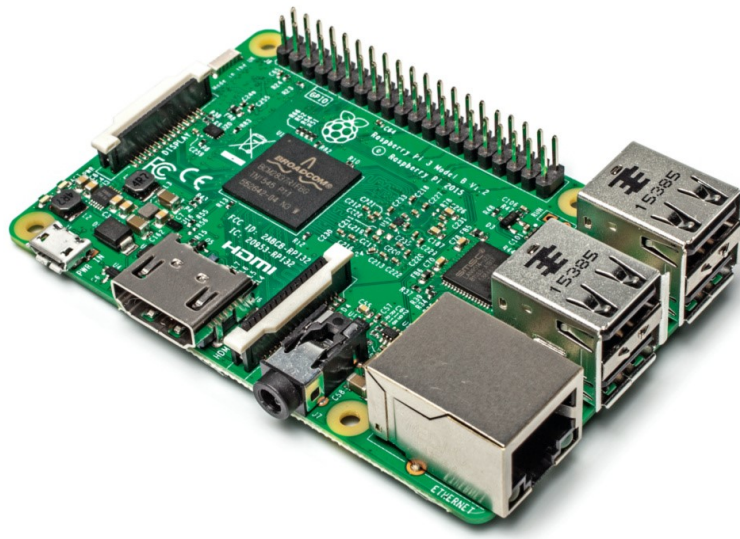
Example: ARM SoC

SGM-775



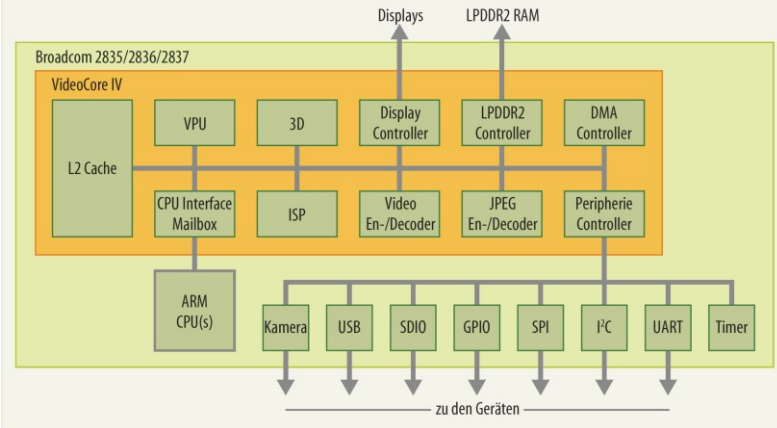
Example: Broadcom BCM2837B0 SoC

- ARMv8-A (32-bit/64-bit) architecture
- ARM Cortex-A53 quad-core processor (@ 1.4 GHz)
- 1GB RAM
- Assembled in Raspberry Pi 3 B+



Schematischer Aufbau des Raspberry Pi

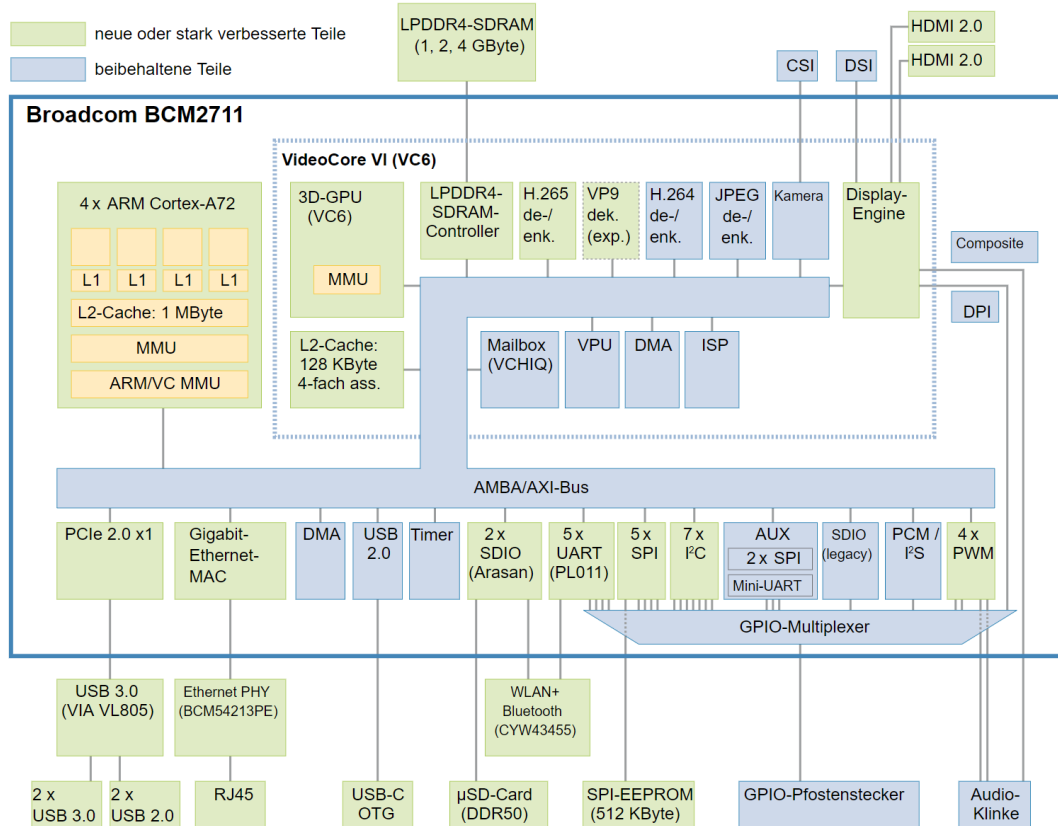
Anders als man erwarten könnte, führt bei den Raspis nicht die ARM-CPU, sondern der Multimediaprozessor VideoCore das Regiment – und ist gleichzeitig der limitierende Faktor.



Example: Raspberry Pi 4

Herz des Raspberry Pi 4: Broadcom BCM2711

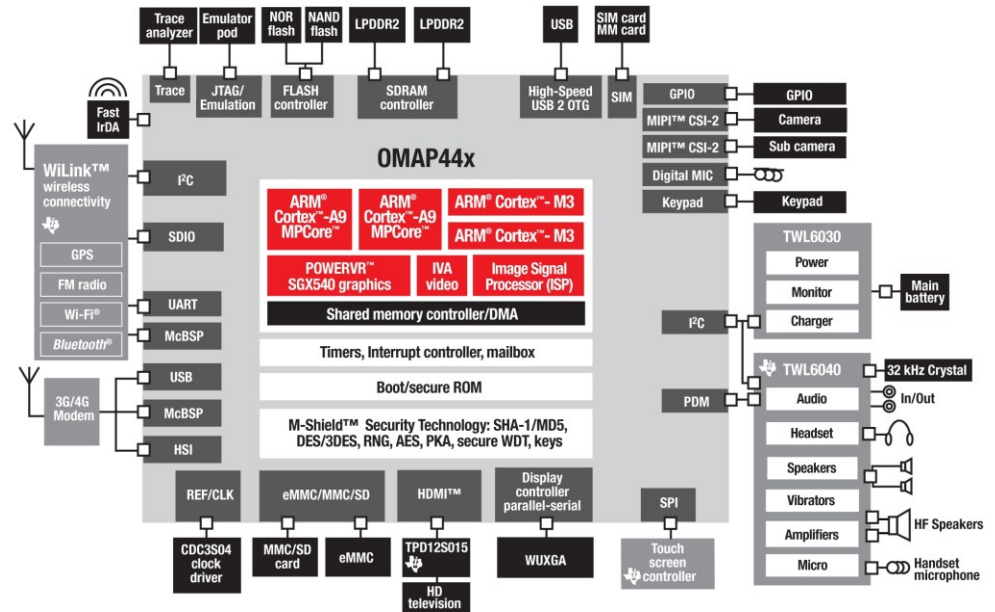
Das System-on-Chip (SoC) BCM2711 vereint nicht nur vier CPU-Kerne mit einer GPU, sondern enthält auch Controller für viele Schnittstellen.



©1 Magazin für Computertechnik 2019

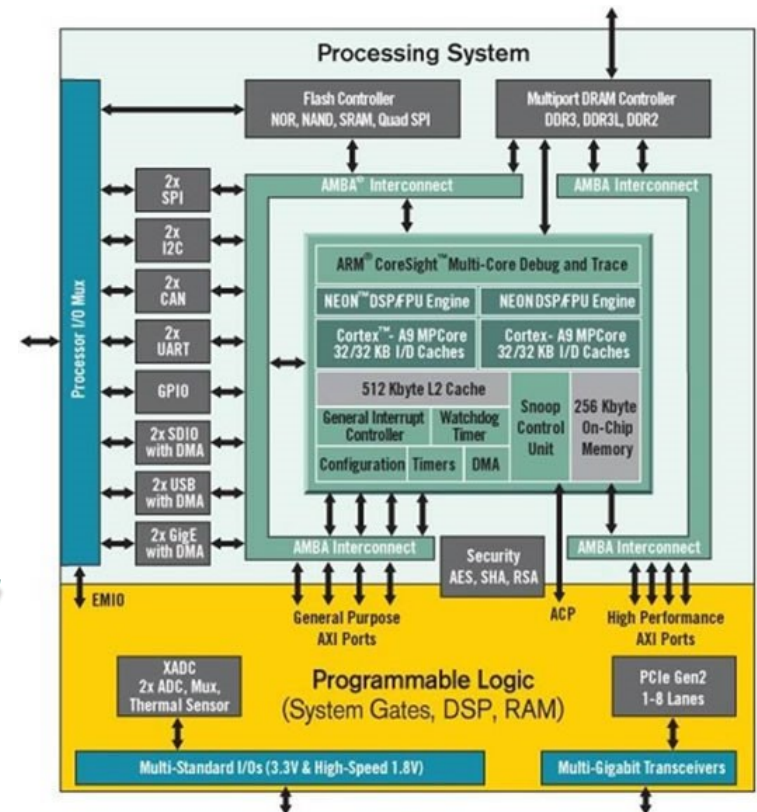
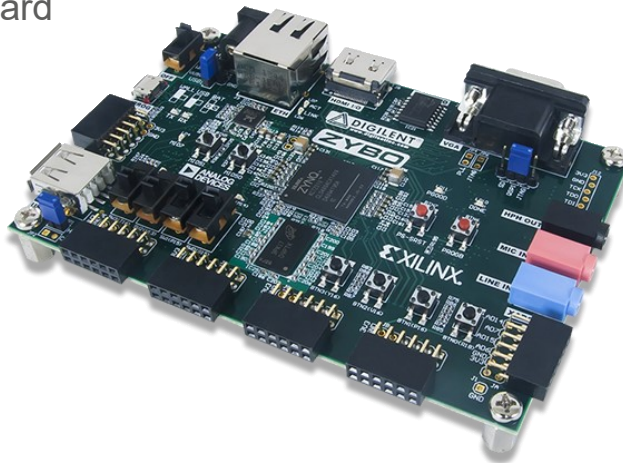
Example: TI OMAP44x SoC

- ARMv7-A (32-bit) architecture
- ARM Cortex-A9 dual-core processor (@1.2-1.5 GHz)
- Co-processor dual Cortex-M3 (@266 MHz)
- Assembled in Pandaboard ES, ...



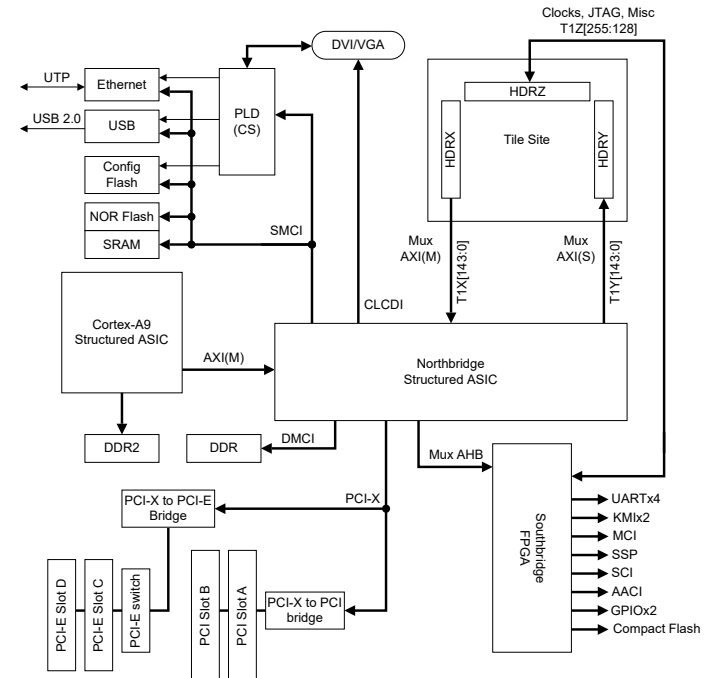
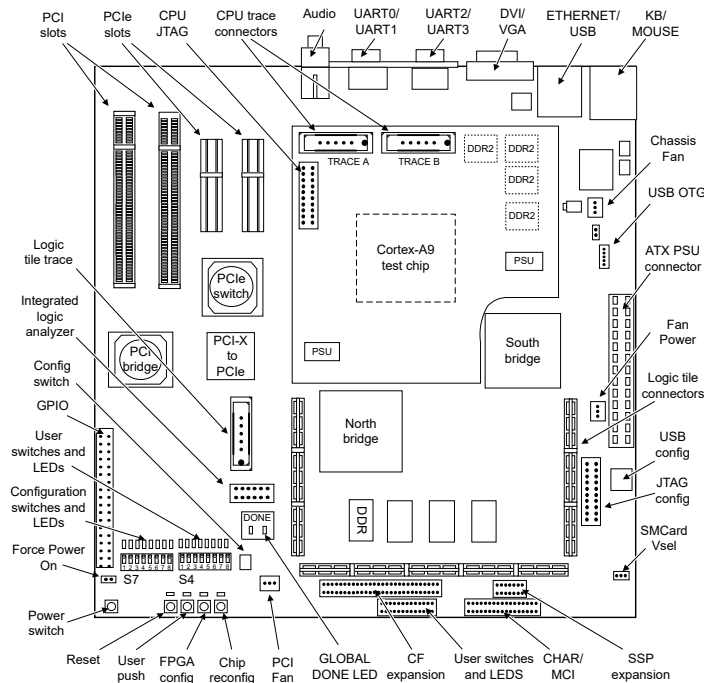
Example: Xilinx Zynq-7000 SoC

- ARMv7-A (32-bit) architecture
- Processing System (PS): ARM Cortex-A9 dual-core processor (up to 1 GHz)
- Programmable Logic (PL): Artix-7 or Kintex-7 FPGA
- Assembled in
 - Digilent Zybo
 - Avnet Zedboard
 - ...

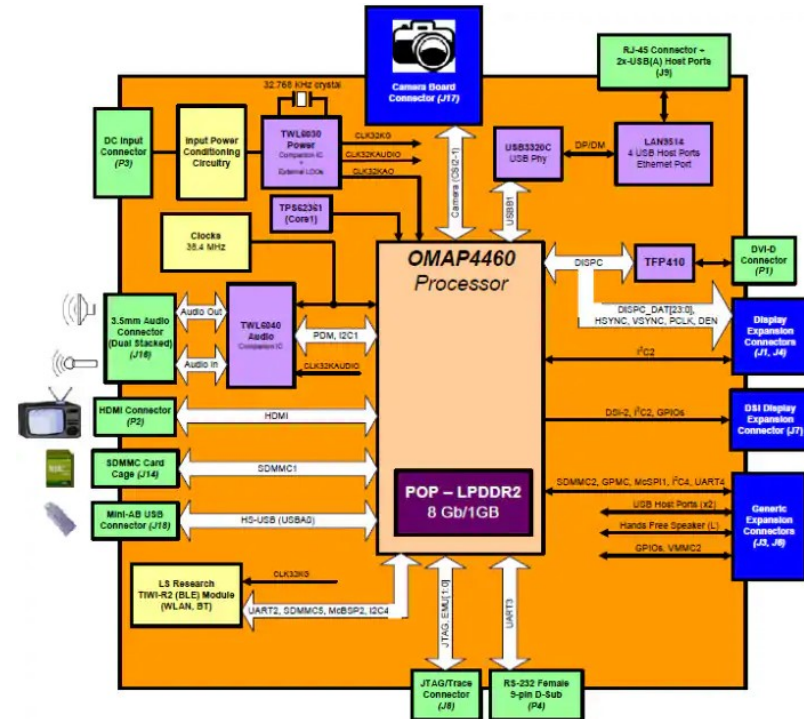
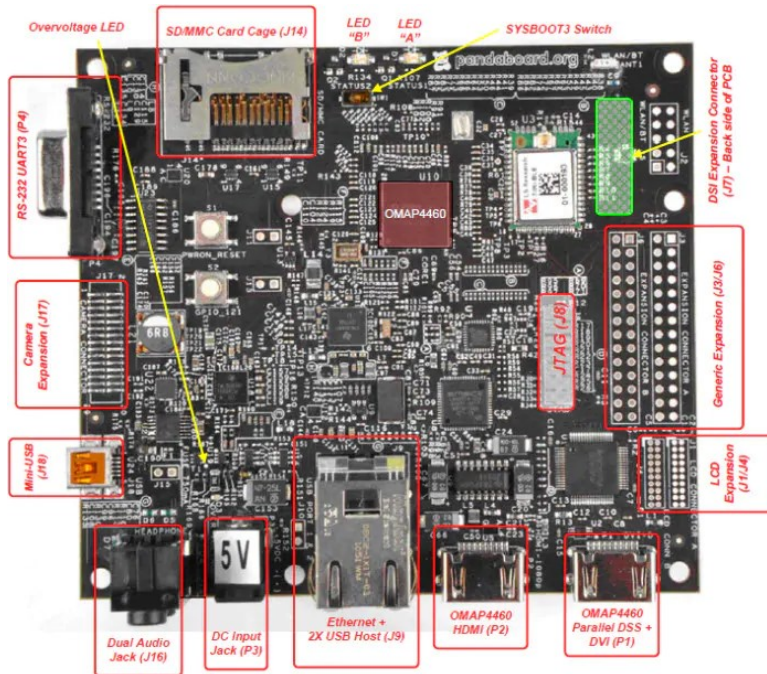


SBC

ARM Reference Platform – ARM Realview PBX-A9



Example SBC: Pandaboard ES



Bootloader

Bootloader - Definition

- A bootloader is a piece of software, which is executed after the on-chip firmware and is responsible for loading the operating system
- Classical PC
 - After starting the PC, a built-in program is run -> today mostly UEFI (in the past BIOS)
 - Self-tests and menu configuration
 - Loads bootloader (Windows Boot Manager, Grub, rEFInd, ...)
 - Multi-stage bootloaders possible (rEFInd loads Windows Boot Manager, loads Windows)
 - Bootloader boots the operating systems
- Embedded Devices
 - Normally no BIOS or UEFI – FSBL (first stage bootloader) in ROM
 - Initializes the RAM and loads second stage bootloader into RAM
 - Second stage bootloader loads the kernel into RAM
 - Kernel gets executed

Excursus: BIOS/UEFI <-> Libreboot

- Libreboot
 - Free software
 - BIOS/UEFI replacement
 - Initialize HW and boot operating system
 - Is a coreboot distribution without proprietary SW (Intel Mgmt Engine, VGA BIOS, ...)
- Goals
 - Recommend and distribute only free SW
 - Support as much hardware as possible
 - Make coreboot easy to use
- Beware
 - One still needs a bootloader like GRUB, ...
 - Limited HW support: mostly Thinkpad laptops, Apple MacBooks, a few x86 mainboards, ...

Bootloader – U(niversal)-Boot

- Free and open source SW
- Used as the default (second stage) bootloader by several board vendors
- As of today support for many different architectures (x86_64, arm, arm64, ...)
- Board support for SoC and specific (but sometimes limited) device drivers
- Design principles
 - Should be easy to port to new architecture, new processors, new boards
 - Be as small and reliable as possible
- Current version is v2020.10

```

U-Boot 2010.12-xes_r3 (Aug 25 2011 - 11:04:04)

CPU0: P2020E, Version: 1.0, (0x80ea0010)
Core: E500, Version: 4.0, (0x80211040)
Clock Configuration:
  CPU0:1066.667 MHz, CPU1:1066.667 MHz,
  CCB:533.333 MHz,
  DDR:400 MHz (800 MT/s data rate) (Asynchronous), LBC:133.333 MHz
L1:  D-cache 32 kB enabled
     I-cache 32 kB enabled
Board: X-ES XPedite501 PMC/PMC SBC
      Rev 5A, Serial# 36093001, Cfg 90015130-1
I2C:  ready
DRAM:  2 GiB (DDR3, 64-bit, CL=6, ECC on)
FLASH: Executed from FLASH1
POST memory PASSED
FLASH: 256 MiB
L2:    512 KB enabled
NAND:  4096 MiB
PCIE1: connected as Root Complex (no link)
PCIE1: Bus 00 - 00
PCIE2: disabled
PCIE3: disabled
In:     serial
Out:    serial
Err:    serial
DTT:    37 C local / 59 C remote (adt7461@4c)
DTT:    37 C local (lm75@48)
Net:    eTSEC1 connected to Broadcom BCM5402S
        eTSEC2 connected to Broadcom BCM5482S
        eTSEC1, eTSEC2
POST i2c PASSED
Hit any key to stop autoboot: 0
=>

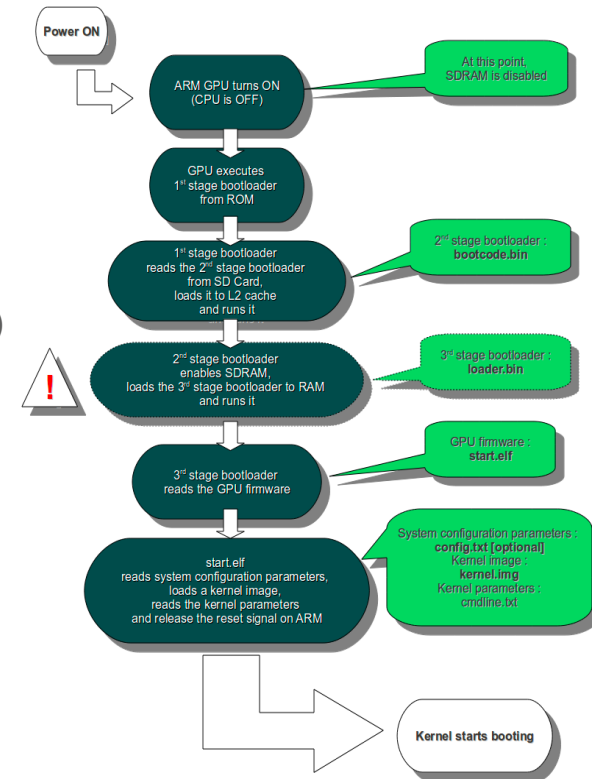
```

U-Boot - Features

- Upgrade features (self-upgrade, e.g. when stored in an on-board flash memory)
- Networking support (DHCP, TFTP, ...)
- Autoboot
- Scripting Environment
- Access different memory devices/flash storage
- Program/Initialize co-processors (FPGA, ...)

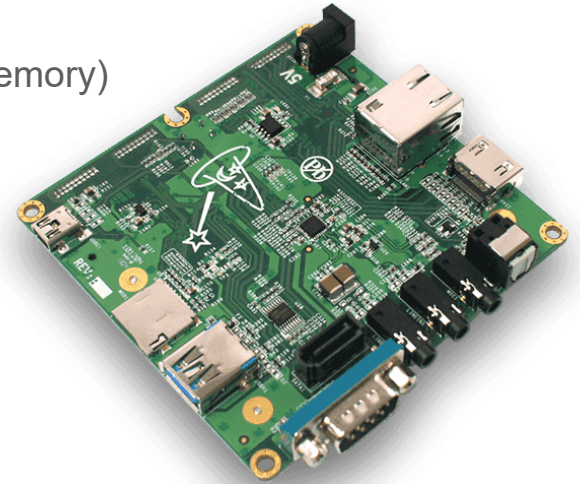
Example: Raspberry Pi

- VideoCore: first active component and boots the system
 - There are some binary blobs needed for a successful boot
 - Since they are binary pieces of SW, no source code available (but there exists an open firmware project)
- Boot flow
 1. Code from on-chip ROM loads „bootcode.bin“ (stage 2 bootloader) from FAT formatted SD card into L2 cache
 2. „bootcode.bin“ gets executed on the GPU, activates the SDRAM and loads stage 3 bootloader „start.elf“ from SD card
 3. „start.elf“ reads firmware configuration (config.txt), splits the RAM between GPU and ARM CPU, enables ARM CPU and loads the stage 4 bootloader „u-boot.bin“
 4. U-Boot loads the OS image



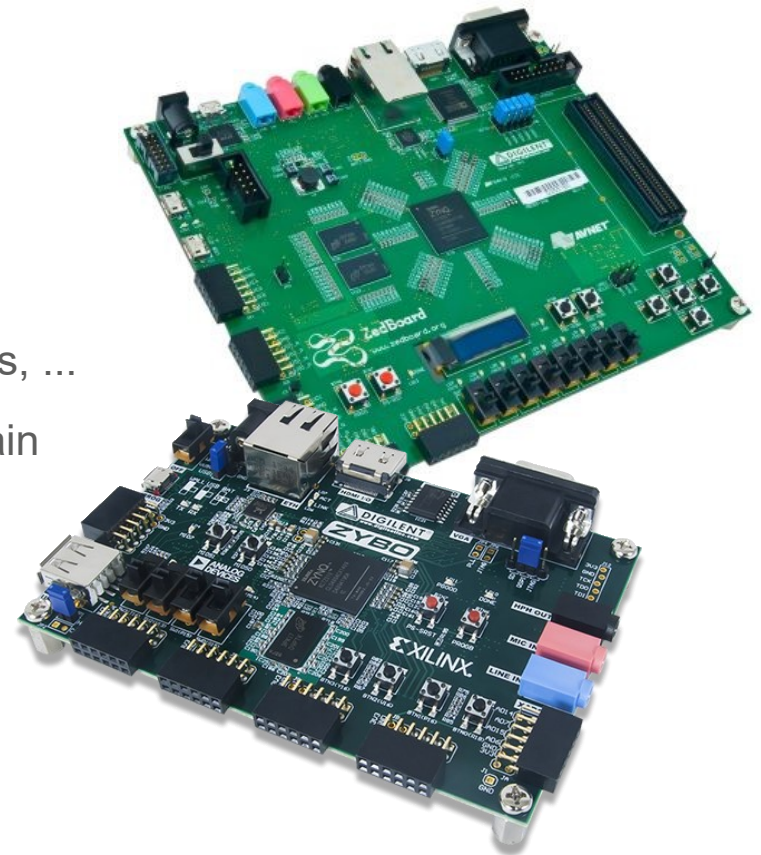
Example: Wandboard

1. ROM code (hardcoded into the i.MX6 CPU)
 1. Reads the table of memory parameters inside U-Boot (sets up memory)
 2. Loads bootloader (from SD card) into RAM
 2. Bootloader loads kernel into RAM
 3. Kernel gets executed
- Beware:
 - ROM code doesn't know about file systems/partitions and expects bootloader at specific address on the SD card
 - Optional: Secondary Program Loader (SPL) as a very small bootloader, able to fit into the very small amount of memory (128KB) available inside the CPU
 - Autodetection of configuration (e.g. SOLO, DUAL, QUAD and sets up memory accordingly)
 - After memory is set up, SPL loads U-Boot and the booting proceeds as usual



Example: Zybo/Zedboard

- What the FSBL (First Stage Bootloader) does:
 - Configure FPGA with bitstream file (optional)
 - Load operating system (or 2nd stage bootloader)
 - Take CPU out of reset
- U-Boot SPL can replace Xilinx FSBL
- „Missing“ some features, like encrypted bitstream files, ...
- Beware: Needs to be created with proprietary toolchain



Sources

- p. 4
 - <https://pics.computerbase.de/9/5/8/0/2-e657c12372672be2/article-630x354.e02a302c.jpg>
 - <https://www.nvidia.com/de-de/self-driving-cars/drive-platform/hardware/>
 - Arm_SB_Q3_2017_Roadshow_Slides_Final
 - <https://www.heise.de/newsticker/meldung/Computex-ARM-Server-mit-Cavium-ThunderX-im-Herbst-2672140.html>
- p. 5 & 6
 - <https://community.arm.com/processors/b/blog/posts/a-brief-history-of-arm-part-2>
- p. 9
 - <https://www.allaboutcircuits.com/technical-articles/arm-architecture-explained/>
- p. 10
 - https://courses.cs.washington.edu/courses/cse466/10au/pdfs/lectures/07-arm_overview.pdf
 - <https://www.slideshare.net/AnhDungNGUYEN3/aame-tc2013-001v02-architecture-and-programmers-model>

Sources ctd.

- p. 12
 - <https://www.heise.de/imgs/18/7/3/0/2/3/6/ARMv8.jpg-a34e91dd69c26b1d.jpeg>
- p. 13
 - <https://developer.arm.com/ip-products/processors/cortex-a/cortex-a78>
 - <https://developer.arm.com/ip-products/processors/cortex-r/cortex-r82>
 - <https://developer.arm.com/ip-products/processors/cortex-m/cortex-m55>
- p. 14
 - <https://www.arm.com/-/media/global/products/hero-chips/cortex-a55.svg>
- p. 16
 - <https://developer.arm.com/products/processors/cortex-a/cortex-a5>
 - <https://developer.arm.com/products/processors/cortex-a/cortex-a7>

Sources ctd.

- p. 17
 - <https://developer.arm.com/products/processors/cortex-a/cortex-a72>
 - <https://developer.arm.com/products/processors/cortex-a/cortex-a53>
- p. 18
 - <https://www.anandtech.com/show/16584/arm-announces-armv9-architecture/3>
- p. 19 & 20
 - <https://www.anandtech.com/show/16584/arm-announces-armv9-architecture>
- p. 21
 - <https://anyconnect.com/wp-content/uploads/cortex-r52.svg>
- p. 23
 - <https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/cortex-m-resources>

Sources ctd.

- p. 25
 - <https://community.arm.com/cn/b/blog/posts/mcu-114891201>
- p. 26
 - <https://www.elektroniknet.de/halbleiter/mikrocontroller/cortex-m23-und-m33-sind-erste-armv8-m-cpus.135091.3.html>
- p. 27
 - <https://community.arm.com/processors/trustzone-for-armv8-m/f/discussions/8338/what-is-the-top-level-difference-in-features-between-cortex-m33-and-cortex-m4>
- p. 29
 - <https://developer.arm.com/-/media/Arm%20Developer%20Community/Images/Block%20Diagrams/Neoverse%20Processor/Neoverse%20N1%20Hyperscale%20Reference%20Design.jpg>
 - <https://developer.arm.com/-/media/Arm%20Developer%20Community/Images/Block%20Diagrams/Neoverse%20Processor/Neoverse%20N1%20Edge%20Reference%20Design.JPG>
 - <https://developer.arm.com/-/media/Arm%20Developer%20Community/Images/Block%20Diagrams/Neoverse%20Processor/Neoverse%20E1%20Edge%20Reference%20Design.JPG>

Sources ctd.

- p. 31
 - <https://developer.arm.com/products/processors/cortex-m/sc300-processor>
 - <https://developer.arm.com/products/processors/cortex-m/sc000-processor>
 - <https://developer.arm.com/ip-products/processors/machine-learning/arm-ethos-n/ethos-n78>
- p. 32
 - <https://developer.arm.com/ip-products/processors/machine-learning/arm-nn>
- p. 33
 - http://web.cs.wpi.edu/~cs4515/d15/Protected/LecturesNotes_D15/CS4515-TeamB-Report.pdf
- p. 34
 - <https://community.arm.com/developer/ip-products/processors/b/processors-ip-blog/posts/where-does-big-little-fit-in-the-world-of-dynamiq>
 - <https://www.linaro.org/blog/big-little-software-update/>

Sources ctd.

- p. 36
 - <https://community.arm.com/developer/tools-software/oss-platforms/w/docs/489/system-guidance-for-infrastructure-sgi>
- p. 38
 - <https://developer.arm.com/-/media/Arm%20Developer%20Community/Images/Block%20Diagrams/Corstone/SSE-050-block-diagram.png>
 - <https://developer.arm.com/-/media/Arm%20Developer%20Community/Images/Block%20Diagrams/Cortex%20Microcontroller%20Software%20Interface%20Standard%20-%20CMSIS/CMSDK-M3-M4-block-diagram.png>
- p. 39
 - <https://community.arm.com/developer/tools-software/tools/b/tools-software-ides-blog/posts/using-ds-5-trace-with-arm-fast-models>
- p. 40
 - <https://www.arm.com/zh/assets/images/CoreSight-call-outs.png>
 - https://www.arm.com/assets/images/Diagram_Large.png

Sources ctd.

- p. 43
 - <https://developer.arm.com/tools-and-software/development-boards/iot-test-chips-and-boards/musca-a-test-chip-board>
 - <https://developer.arm.com/tools-and-software/development-boards/iot-test-chips-and-boards/beetle-iot-evaluation-platform>
- p. 45
 - <https://riscv.org/wp-content/uploads/2018/09/riscv-logo-1.png>
 - <https://www.anandtech.com/show/10488/sifive-unveils-freedom-platforms-for-riscvbased-semicustom-chips>
- p. 46
 - <https://www.crowdsupply.com/sifive/hifive-unmatched>
 - <https://www.crowdsupply.com/microchip/polarfire-soc-icicle-kit>
- p. 49
 - https://en.wikipedia.org/wiki/System_on_a_chip

Sources ctd.

- p. 50
 - <https://community.arm.com/developer/tools-software/oss-platforms/w/docs/469/system-guidance-for-mobile-sgm>
- p. 51
 - <https://www.heise.de/select/ct/2016/8/1460193213013079>
 - https://heise.cloudimg.io/bound/500x500/q60.png-lossy-60.webp-lossy-60.foil1/_www-heise-de_/select/ct/2016/8/1460193213013079/tn_Raspi_Broadcom_mtk_IG.jpg
- p. 52
 - https://heise.cloudimg.io/width/1566/q50.png-lossy-50.webp-lossy-50.foil1/_www-heise-de_/ct/imgs/04/2/7/4/3/7/8/1/RPi4-Block-e4359fdb4be588cf.png
- p. 53
 - <https://embeddedboard.files.wordpress.com/2011/11/omap4460-block-diagram1.jpg>
- p. 54
 - <https://www.digikey.de/de/product-highlight/x/xilinx/zynq-7-soc>
 - https://reference.digilentinc.com/_media/reference/programmable-logic/zybo/zybo-0.png

Sources ctd.

- p. 56
 - <https://developer.arm.com/documentation/dui0440/b/hardware-description/baseboard-architecture>
 - <https://developer.arm.com/documentation/dui0440/b/hardware-description/baseboard-architecture/system-architecture>
- p. 58
 - <https://www.mouser.de/new/pandaboardorg/pandaboardES/>
- p. 63
 - <https://raspberrypi.stackexchange.com/questions/10442/what-is-the-boot-sequence>
- p. 64
 - <https://www.wandboard.org/>
- p. 65
 - <https://at.rs-online.com/web/p/entwicklungskits-prozessor-mikrocontroller/1346457/>

HENSOLDT Cyber GmbH
Willy-Messerschmitt-Straße 3
82024 Taufkirchen
www.hensoldt-cyber.com

